

School of Computer Science and Engineering
(Computer Science & Engineering - AIML)

Faculty of Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112 Ramanagara District, Karnataka, India

2025-2026

(VII Semester)

A Project Report on

**“FACE RECOGNITION AND DEEP FAKE
DETECTION”**

FACE RECOGNITION AND DEEP FAKE DETECTION SYSTEM

Submitted in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY IN
COMPUTER SCIENCE & ENGINEERING- AIML

Submitted by

M.SAI SHREYAS REDDY (22BTRCL094)

C.VBHARGAV GOWD (22BTRCL043)

K.HARSHA VARDHAN (22BTRCL075)

C.MADHU KEERTHI (22BTRCL038)

Under the guidance of

Dr.KANITHAN.S,

Associate Professor,

**Department of Computer Science & Engineering – AIML,
School of Computer Science & Engineering Faculty of Engineering & Technology,
JAIN (Deemed to-be University).**

Department of Computer Science & Engineering - AIML

School of Computer Science & Engineering Faculty of
Engineering & Technology

Jain Global Campus, Kanakapura Taluk - 562112 Ramanagara District, Karnataka, India

CERTIFICATE

This is to certify that the project work titled **“FACE RECOGNITION AND DEEP FAKE DETECTION”** is carried out by **M.SAI SHREYAS REDDY(22BTRCL094), C.VBHARGAV GOWD(22BTRCL043), K.HARSHA VARDHAN (22BTRCL075), C.MADHU KEERTHI (22BTRCL038)** a Bonafide students of Bachelor of Technology at the School of Computer Science and Engineering - AIML , Faculty of Engineering & Technology, JAIN (Deemed-to-be University), Bangalore in partial fulfillment for the award of degree in Bachelor Technology in Computer Science & Engineering - AIML, during the year **2024-2025**.

Dr.KANITHAN.S

Project Guide ,
Computer Engineering, Software
Engineering - AIML
School of Computer Science and
Engineering
Faculty of Engineering &
Technology
JAIN (Deemed to-be University)
Date:

Dr. Chandrasekar.V

Program Head,
Computer Engineering, Software
Engineering - AIML
School of Computer Science and
Engineering
Faculty of Engineering &
Technology
JAIN (Deemed to-be University)
Date:

Dr. Kamlesh Tiwari

Head of Department,
Computer Science &
Engineering
Faculty of Engineering &
Technology
JAIN (Deemed to-be
University)
Date:

Name of the Examiner

Signature of Examiner 1.

Examiner 2.

DECLARATION

We, M.SAI SHREYAS REDDY(22BTRCL094), C.VBHARGAV GOWD(22BTRCL043), K.HARSHA VARDHAN (22BTRCL075), C.MADHU KEERTHI (22BTRCL038) student of 8TH semester B.Tech in Computer Science & Engineering - AIML, at School of Engineering & Technology, Faculty of Engineering & Technology, JAIN (Deemed to - be University), hereby declare that the Project work titled “FACE RECOGNITION AND DEEP FAKE DETECTION” FACE RECOGNITION AND DEEP FAKE DETECTION Detection System has been carried out by us and submitted in partial fulfilment for the award of degree in Bachelor of Technology in Computer Science & Engineering - AIML during the academic year 2025-2026. Further, the matter presented in the work has not been submitted previously by anybody for the award of any degree or any diploma to any other University, to the best of our knowledge and faith.

**M.SAI SHREYAS REDDY:
(22BTRCL094)**

**C.V.BHARGAV GOWD:
(22BTRCL043):**

**K.HARSHA VARDHAN:
(22BTRCL075)**

**C.MADHU KEERTHI:
(22BTRCL038)**

Place: Bangalore

Date:

ACKNOWLEDGEMENT

It is a great pleasure for me to acknowledge the assistance and support of a large number of individuals who have been responsible for the successful completion of this project work.

First, I take this opportunity to express my sincere gratitude to Faculty of Engineering & Technology, JAIN (Deemed to-be University) for providing me with a great opportunity to pursue my Bachelor's Degree in this institution.

*I am deeply thankful to several individuals whose invaluable contributions have made this project a reality. I wish to extend my heartfelt gratitude to **Dr. Chenraj Roy Chand, Chancellor**, for his tireless commitment to fostering excellence in teaching and research at Jain (Deemed-to-be-University). I am also profoundly grateful to the honorable **Dr. Dinesh Nilkant, Pro Vice Chancellor**, for their unwavering support. Furthermore, I would like to express my sincere thanks to **Dr. Jitendra Kumar Mishra, Registrar**, whose guidance has imparted invaluable qualities and skills that will serve us well in our future endeavors.*

*I extend my sincere gratitude to **Dr. Kamlesh Tiwari, Head of Department** of the School of Computer Science & Engineering within the Faculty of Engineering & Technology, for their constant encouragement and expert advice. Additionally, I would like to express my appreciation to **Dr. Deepak Kumar Sinha, Deputy Director (Students & Industry Relations)**, for their invaluable contributions and support throughout this project.*

*It is a matter of immense pleasure to express my sincere thanks to **Dr. Chandrasekar.V, Program Head, Computer Science & Engineering - AIML**, School of Computer Science & Engineering Faculty of Engineering & Technology for providing right academic guidance that made my task possible.*

*I would like to thank our guide **Dr.KANITHAN.S, Associate Professor, Dept. of Computer Science & Engineering - AIML**, for sparing his/her valuable time to extend help in every step of my work, which paved the way for smooth progress and fruitful culmination of the project.*

*I would like to thank our **Project Coordinator Dr. K.S.Arvind**, and all the staff members of Computer Science & Engineering - AIML for their support.*

I am also grateful to my family and friends who provided me with every requirement throughout the course.

I would like to thank one and all who directly or indirectly helped me in completing the work successfully.

Signature of Student(s)

M.SAI SHREYAS REDDY
22BTRCL094

C.V.BHARGAV GOWD
22BTRCL043

K.HARSHA VARDHAN
22BTRCL075

C.MADHU KEERTHI
22BTRCL038

TABLE OF CONTENTS

CERTIFICATE	i-ii
DECLARATION	iii
ACKNOWLEDGEMENT	iv-v
TABLE OF CONTENTS	vi-vii
LIST OF FIGURES	viii
LIST OF TABLE(S)	viii
ABSTRACT	ix

Chapter No.	Chapter Title	Page Number(s)
Chapter 1	Introduction	10
1.1	Background & Motivation	10
1.2	Objectives	11
1.3	Benefits of Research	12
Chapter 2	Literature Survey	13-15
2.1	Literature Review	15
2.1.1	Analysis of Literature	16
2.2	Problem Formulation and Proposed Work	17-18
2.2.1	Introduction	17-18
2.2.2	Problem Statement	18
2.2.3	System Architecture / Model	19
2.2.4	Proposed Algorithms	20
2.2.5	Proposed Work	20-21
Chapter 3	System Analysis and Proposed Methodology	22
3.1	Introduction	22
3.2	Problem Statement	22-23
3.3	System Architecture / Model	23
3.4	Proposed Algorithms	24
3.5	Proposed Work	25-26
Chapter 4	System Design and Implementation	27
4.1	Introduction	27
4.2	System Design Principles	28-29
4.3	System Architecture	30-31
4.4	Dataset Design & Preprocessing	32
4.4.1	Dataset Structure	33
4.4.2	Preprocessing Techniques	34
4.4.3	Data Augmentation	34
4.5	Model Architecture & Implementation	34
4.5.1	EfficientNetV2B0 Backbone	34
4.5.2	Custom Classification Head	34
4.5.3	Training Mechanism	34
4.5.4	Training Callbacks	34
4.6	Streamlit Application Design	35
4.6.1	User Interface Modules	35
4.6.2	Real-Time Prediction Flow	36
4.6.3	Error Handling	36
4.7	Implementation Workflow	36-37

Chapter 5	Tools and Technologies Required	38
5.1	Development Tools	38
5.2	Frontend Technologies	39
5.3	Backend Technologies	39
5.4	Deployment & Hosting Tools	40
5.5	Testing & Monitoring Tools	40-41
5.6	Optional / Future Integration Tools	41
Chapter 6	Software Requirement Specification (SRS)	42
6.1	Specific Requirements	42
6.2	Functional Requirements	42-43
6.3	Non-Functional Requirements	44
6.4	Human-Centered Design Requirements	44-45
6.5	Specific Requirements	45
6.5.1	Software Interfaces	45
6.5.2	Communication Interfaces	46
Chapter 7	Results and Discussion	47
7.1	Introduction	47
7.2	Model Performance Metrics	47
7.2.1	Accuracy	48
7.2.2	Precision and Recall	49-50
7.3	Classification Report Analysis	51-55
7.4	Confusion Matrix Discussion	55-56
7.5	Visualization & Evaluation Insights	57
7.6	Real-Time Deployment Results	57
7.7	Discussion of Findings	58
Chapter 8	Conclusions and Future Scope	59
8.1	Conclusions	59
8.2	Future Scope	60
8.3	Summary	61
References	References	62-63
Information	Student Information	64
Regarding		
Student(s)		

LIST OF FIGURES

S.no	Fig Number	Page Number(s)
1	<i>Fig-4.1 System Architecture Diagram</i>	28
2	<i>Fig-4.2 Architecture Diagram</i>	30
3	<i>Fig-7.1 Interface output</i>	47
4	<i>Fig-7.2 Accuracy Graph</i>	48
5	<i>Fig-7.3 Precision Graph</i>	49
6	<i>Fig-7.4 Recall Graph</i>	50
7	<i>Fig-7.5 Customer Portal (New Claim page)</i>	51
8	<i>Fig-7.6 Customer Portal (Tracking Page)</i>	51
9	<i>Fig-7.7 Customer Portal Tracking Page (Non-fraud Claim)</i>	52
10	<i>Fig-7.8 Customer Portal Tracking Page (fraud Claim)</i>	52
11	<i>Fig-7.9 Business Dashboard (with Pending Requests)</i>	53
12	<i>Fig-7.10 Business Dashboard</i>	53
13	<i>Fig-7.11 real image Output no.2 (Heatmap)</i>	54
14	<i>Fig-7.12 fake image Output no.3(Explainable AI)</i>	54
15	<i>Fig-7.13 fake image Output no.4(LIME)</i>	55
16	<i>Fig-7.14 fake image Output no.5(Risk Score Matrix & Decision)</i>	55

LIST OF TABLE(S)

S.no	Table Number and Name	Page Number(s)
1	<i>Table-2.1 Literature Review</i>	14-15

ABSTRACT

The rapid growth of digital technologies has significantly improved face recognition systems, making them widely used in areas such as security, surveillance, and identity verification. However, this advancement has also led to the rise of deepfake technology, where facial images and videos can be artificially manipulated to create highly realistic but fake content. This poses serious challenges in maintaining trust, authenticity, and security in digital systems. Detecting whether a face is real or manipulated has therefore become an important problem in modern computer vision applications

The proposed system addresses this challenge by combining face recognition with deepfake detection into a unified framework. The system utilizes the MTCNN algorithm for accurate face detection, followed by a feature extraction process that includes techniques such as Histogram of Oriented Gradients (HOG), Local Binary Patterns (LBP), color histograms, and edge-based features. These features are used to generate unique representations of faces, which are then compared using cosine similarity to identify individuals. Alongside recognition, a deepfake detection module analyzes visual inconsistencies such as texture variations, facial symmetry, and edge distortions to determine whether the image is real or artificially generated.

To improve usability and accessibility, the system is implemented as a web-based application using Streamlit, providing an interactive and user-friendly interface. Users can upload images, detect multiple faces, recognize known individuals, and evaluate the authenticity of each face in real time. The system also allows users to store new faces in a structured database, enabling continuous learning and improved recognition performance.

A local file-based storage system is used to manage and organize facial data efficiently, ensuring easy retrieval and updates. Overall, the proposed solution offers a practical, scalable, and efficient approach to real-time face recognition and deepfake detection. It enhances system reliability, reduces the risk of identity spoofing, and supports secure decision-making in applications such as surveillance, authentication systems, and digital forensics.

Chapter 1

1. Introduction

In today's digital world, face recognition technology has become an essential part of many applications, including security systems, surveillance, and identity verification. From unlocking smartphones to monitoring public spaces, facial recognition offers a convenient and efficient way to identify individuals. However, alongside these advancements, the emergence of deepfake technology has introduced serious challenges. Deepfakes are artificially generated or manipulated images and videos that can closely mimic real human faces, making it increasingly difficult to distinguish between genuine and fake content.

With the widespread availability of powerful image editing tools and AI-based generative models, creating realistic fake facial content has become easier than ever. This raises significant concerns in areas such as cybersecurity, digital forensics, and personal privacy. Fake identities, manipulated media, and impersonation attacks can lead to misinformation, fraud, and unauthorized access to secure systems. As a result, relying solely on traditional face recognition systems is no longer sufficient, as they are not designed to verify the authenticity of facial data.

. Conventional face recognition methods primarily focus on identifying individuals based on facial features. While these systems can achieve good accuracy under controlled conditions, they often struggle with variations such as lighting, pose, and image quality. More importantly, they lack the capability to detect whether the input image has been altered or generated artificially. This limitation highlights the need for a more advanced and integrated solution that not only recognizes faces but also ensures that the detected faces are real.

To overcome these challenges, this project presents a **Face Recognition and Deepfake Detection System** that combines identity recognition with authenticity verification. The system uses the MTCNN algorithm for accurate face detection, enabling it to identify multiple faces within a single image..

1.1 Background & Motivation

In recent years, there has been a rapid shift toward digital systems for identity verification, driven by the growing need for secure and efficient authentication methods. Face recognition technology has become one of the most widely adopted solutions due to its convenience and non-intrusive nature. It is now commonly used in applications such as smartphone unlocking, surveillance systems, attendance management, and online identity verification. Despite its widespread adoption, traditional face recognition systems face significant challenges when dealing with manipulated or synthetic facial content. At the same time, advancements in artificial intelligence have led to

the development of deepfake technologies, which can generate highly realistic fake images and videos. These manipulated visuals can closely resemble real individuals, making it increasingly difficult to detect authenticity using conventional methods. This creates serious risks in areas such as cybersecurity, digital identity verification, and misinformation. Unauthorized access, identity spoofing, and fraudulent activities can occur if systems fail to distinguish between real and fake faces.

Existing face recognition systems primarily focus on identifying individuals based on facial features, without verifying whether the input image is genuine. This limitation makes them vulnerable to attacks using deepfake images or manipulated media. As digital platforms continue to grow, the need for a more secure and reliable solution has become essential.

The motivation behind this project is to address these challenges by developing a system that not only recognizes faces but also verifies their authenticity. By integrating deepfake detection with face recognition, the system aims to provide a more robust and trustworthy solution for modern applications.

1.2 Objective

The main objectives of this project are:

1. To design and develop an automated system capable of detecting human faces from images with high accuracy using advanced computer vision techniques.
2. To implement an efficient face recognition mechanism that can identify individuals by extracting and comparing unique facial features.
3. To develop a deepfake detection module that analyzes visual inconsistencies in images and classifies faces as real or fake.
4. To integrate face recognition and deepfake detection into a single unified framework to improve system reliability and security.
5. To utilize feature extraction techniques such as HOG, LBP, color histograms, and edge-based analysis for generating meaningful facial representations.
6. To design and implement a user-friendly web-based application using Streamlit that allows users to upload images and receive real-time analysis results.

7. To create a structured database for storing known faces, enabling efficient data management and improved recognition performance over time.
8. To provide confidence scores for both recognition and deepfake detection results, helping users better understand the system's predictions.

1.3 Benefits of the Research

This project provides several important technical and practical benefits in the domain of face recognition and digital security.

- **Improved Recognition Accuracy**

The use of multiple feature extraction techniques such as HOG, LBP, color histograms, and edge detection helps the system capture detailed facial characteristics. This improves the accuracy of identifying individuals even under varying conditions.

- **Enhanced Security Against Deepfakes**

By integrating deepfake detection with face recognition, the system is able to identify manipulated or artificially generated images. This reduces the risk of identity spoofing and unauthorized access.

- **Real-Time Processing**

The system is designed to provide quick analysis of images, allowing users to detect and verify faces instantly. This makes it suitable for applications where immediate decisions are required.

- **Increased Reliability and Trust**

The dual-layer approach—combining recognition with authenticity verification—ensures that results are more dependable. Users can trust that the system not only identifies faces but also confirms whether they are genuine.

- **Scalability and Flexibility**

The modular design of the system allows it to handle multiple faces and large datasets efficiently. It can also be extended with advanced deep learning models or additional features in the future.

- **Efficient Data Management**

The system maintains a structured database of known faces, enabling easy storage, retrieval, and updates. This supports continuous improvement in recognition performance over time.

- **User-Friendly Interface**

The Streamlit-based application provides a simple and interactive interface. Users can upload images, view results, and manage data.

Chapter 2

2.Literature Survey

2.1 Literature Review

Face recognition and deepfake detection have gained significant attention in recent years due to the rapid growth of artificial intelligence and digital media technologies. As the use of facial recognition systems has increased across various applications, researchers have explored different approaches to improve accuracy, reliability, and security. At the same time, the emergence of deepfake technology has created new challenges, making it essential to develop methods that can not only recognize faces but also verify their authenticity.

Early research in face recognition primarily relied on traditional techniques such as Eigenfaces and Fisherfaces, which used statistical methods to represent facial features. These approaches were simple and effective under controlled conditions but struggled to handle variations in lighting, pose, and facial expressions. Later, feature-based methods such as Histogram of Oriented Gradients (HOG) and Local Binary Patterns (LBP) were introduced to capture texture and structural information, improving recognition performance to some extent. With the advancement of machine learning, more sophisticated approaches were developed to enhance face recognition systems. Researchers began using feature extraction techniques combined with similarity measures to identify individuals more accurately. However, these methods still had limitations when dealing with large datasets and real-world variations.

In recent years, deep learning has transformed the field of face recognition. Convolutional Neural Networks (CNNs) have been widely used to learn complex facial patterns and generate highly discriminative feature representations. Models such as FaceNet and DeepFace have demonstrated high accuracy and are capable of handling large-scale recognition tasks. These models use deep embeddings to represent faces, making comparisons more robust and reliable. At the same time, the rise of deepfake technology has led to increased research in detecting manipulated facial content. Deepfakes are typically generated using advanced techniques such as Generative Adversarial Networks (GANs), which can produce highly realistic images and videos. To counter this, researchers have proposed various detection methods based on visual inconsistencies, including texture irregularities, unnatural facial symmetry, and frequency-domain artifacts. Deep learning-based classifiers have also been used to distinguish between real and fake images by learning subtle patterns that are difficult to detect manually.

S.No	Paper/Study Title	Authors / Source	Year	Key Focus / Contribution
1	<i>face recognition and deepfake detection in computer vision</i>	Neeta Verma et al.	2023	focused on using deep learning and computer vision techniques for image-based analysis. Their work proposed a Convolutional Neural Network (CNN)..
2	<i>EfficientNetV2 Architecture</i>	Mingxing Tan, Quoc Le	2021	designed to achieve higher accuracy while reducing training time and computational cost.
3	<i>Cost-Sensitive Learning Methods for Imbalanced Data</i>	(IEEE authors from provided abstract)	2010	- Explored hybrid methods combining resampling and cost-sensitive learning. - Emphasized reducing false negatives on minority classes.
4	<i>Cost-Sensitive Learning for Imbalanced Classification</i>	Jason Brownlee	2022	- Practical guide on handling imbalanced datasets. - Described techniques like class weighting, oversampling, threshold tuning.
5	<i>deep learning-based framework that integrates image analysis</i>	L. Maiano et al. — <i>ScienceDirect</i>	2023	framework that integrates image analysis with similarity learning to detect manipulated or duplicate content.
6	<i>Machine Learning-Driven Detection of Fraudulent Vehicle Insurance Claims</i>	Njeru & Miriti — <i>Int. Journal of Computer Applications</i>	2025	Evaluates multiple ML classifiers on balanced/unbalanced fraud data; automates classification.
7	<i>machine learning and deep learning</i>	G. Schrijver — <i>ScienceDirect</i>	2024	solving complex detection problems, which are highly

	<i>techniques</i>			relevant to face recognition and deepfake detection systems.
--	-------------------	--	--	--

8	<i>Concept of multimodal learning</i>	(MIT Digital Intelligence journal)	2020	where multiple data sources such as visual information, textual data, and engineered features are combined to improve detection performance.
9	<i>compared several machine learning algorithms</i>	(ResearchGate)	2025	including AdaBoost, Support Vector Machines (SVM), Artificial Neural Networks (ANN), and XGBoost..
10	<i>issue of data imbalance</i>	P. Komsrimorakot et al. — <i>Springer</i>	2025	proposed hybrid resampling techniques that combine oversampling and undersampling methods. This concept is important in detection systems where certain classes (such as fake images) may be less common but require accurate identification.
11	<i>interpretable Artificial Neural Network using SHAP</i>	A. Wilson — <i>JITP</i>	2024	This work emphasizes the importance of transparency in AI systems, ensuring that users can understand how decisions are made
12	<i>application of various deep learning models</i>	P. Anand Kumar — <i>Nature Scientific Reports</i>	2025	including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Autoencoders, and Graph Neural Networks (GNNs), for complex pattern recognition tasks.

Table-2.1 Literature Review

2.1.1 Analysis of Literature

The review of existing research in the fields of face recognition and deepfake detection reveals several important trends and technological advancements.

- **Significance of Efficient Model Architectures**

Research has shown that efficient model designs play a key role in achieving better performance with reduced computational cost. Architectures such as EfficientNet and its variants are known for providing a balance between accuracy and speed. This makes them suitable for real-time applications, where both performance and resource efficiency are important.

- **Growing Use of Deep Learning for Image Analysis**

Many recent studies highlight the effectiveness of deep learning models, especially Convolutional Neural Networks (CNNs), in analyzing facial images. These models are capable of learning complex patterns and features directly from data, resulting in higher accuracy compared to traditional methods. Transfer learning techniques further enhance performance by allowing models to leverage pre-trained knowledge, reducing training time and improving generalization.

- **Challenges in Handling Variations and Data Imbalance**

Face recognition and deepfake detection systems often deal with variations in lighting, pose, and image quality. Additionally, datasets may not always be balanced, especially when fake samples are limited. To address this, researchers have proposed techniques such as data augmentation, weighted learning, and careful feature selection to improve model robustness and reduce misclassification.

- **Integration of Multiple Feature Extraction Techniques**

Several studies emphasize the importance of combining different types of features to improve system performance. Techniques such as HOG, LBP, color histograms, and edge detection capture different aspects of facial information. Integrating these features helps in building a more reliable representation of faces, leading to better recognition and detection results.

2.2 PROBLEM FORMULATION AND PROPOSED WORK

2.2.1 Introduction

The rapid advancement of digital technologies has made face recognition systems widely used in applications such as security, surveillance, and identity verification. However, the emergence of deepfake technology has introduced a significant challenge, as it enables the creation of highly realistic but manipulated facial images and videos. This makes it difficult to rely solely on traditional recognition systems, which are primarily designed to identify individuals and not to verify the authenticity of the input data.

Conventional systems often depend on single-purpose solutions, either focusing only on face recognition or only on detecting manipulated media. Such approaches lack the ability to provide a comprehensive solution that ensures both identity verification and authenticity validation. Additionally, many systems are not optimized for real-time usage and may struggle to handle multiple faces or varying image conditions effectively.

- Computer vision techniques for accurate face detection
- Feature extraction methods for reliable face recognition
- Image analysis techniques to identify visual inconsistencies associated with deepfake content

This integrated approach ensures that the system not only identifies individuals but also verifies whether the detected faces are genuine.

2.2.2 Problem Statement

The main problem addressed this project is:

To design and develop an intelligent system that can automatically detect faces, recognize individuals, and determine whether the detected faces are real or artificially manipulated, ensuring accuracy, reliability, and real-time performance.

Key challenges include:

- **Detection of Faces in Diverse Conditions**
Accurately identifying faces in images with variations in lighting, pose, background, and image quality.
- **Reliable Face Recognition**
Extracting meaningful features that can uniquely represent individuals and enable accurate

matching with stored data.

- **Detection of Deepfake Content**

Identifying subtle visual inconsistencies in manipulated images, such as unnatural textures, symmetry differences, and edge distortions. Providing explainable results to ensure trust and transparency in predictions

- **Handling Limited and Unbalanced Data**

Managing scenarios where real images are more common than fake ones, which can affect detection performance.

- **Integration of Multiple Functionalities**

Combining face detection, recognition, and deepfake detection into a single cohesive system.

- **Ensuring Real-Time Performance**

Providing fast and efficient processing so that users can obtain results instantly.

2.2.3 System Architecture / Model

The proposed system is designed using a modular architecture, where each component performs a specific task while working together as part of a unified pipeline .key components:

1. User Interface Layer (Streamlit Application)

- Interactive and easy-to-use platform for users.
- It also supports functionalities such as viewing results

2. Face Detection Module

- This module is responsible for identifying and extracting faces from input images.
- It uses the MTCNN algorithm, which is effective in detecting faces
- The module can detect multiple faces in a single image

3. Feature Extraction Module

- Once faces are detected, the system extracts important features that uniquely represent each face.
- This is achieved using a combination of techniques such as Histogram of Oriented Gradients (HOG)

4. Face Recognition Module

- he extracted features are compared with stored data in the database to identify individuals.
- The system uses cosine similarity to measure the similarity between feature vectors.

5. Deep fake Detection Module

- In addition to recognition, the system includes a module to verify whether the detected face is real or manipulated.
- This module analyzes visual inconsistencies such as unnatural textures, facial symmetry differences, and irregular edge patterns.

6. Database

- **The system maintains a structured database to store information about known individuals.**
- **Each person has a dedicated folder containing multiple images, which helps improve recognition accuracy.**

Overall, the proposed architecture provides a well-organized and efficient framework for integrating face recognition with deepfake detection.

2.2.4 Proposed Algorithms

The system integrates multiple algorithms and techniques:

1. MTCNN (Face Detection Algorithm)

- The system uses the Multi-task Cascaded Convolutional Neural Network (MTCNN) to detect faces
- This algorithm is capable of identifying multiple faces under different conditions such as variations in lighting, pose, and background.

2. Feature Extraction Techniques (HOG, LBP, and Color Features)

- HOG captures structural and edge information
- LBP focuses on texture patterns
- Color histograms represent color distribution

3. Cosine Similarity for Face Recognition

- The extracted features are compared with stored face data using cosine similarity.
- Based on this, the system assigns a label along with a confidence score.

4. Deepfake Detection Algorithm

- The system includes a lightweight detection approach that analyzes visual inconsistencies in facial images
- Based on these characteristics, the system classifies the face as either REAL or FAKE.

5. Confidence Scoring Mechanism

- Both recognition and deepfake detection modules generate confidence scores.
- This helps users better interpret the results.

2.2.5 Proposed Work

The proposed work focuses on developing a **comprehensive, end-to-end fraud detection platform** with the following key functionalities:

1. User Interaction Module

- **The system provides an interface where users can upload images and view results instantly.**

2. Face Detection and Recognition Module

- **The system detects all faces present in an image and identifies known individuals by comparing extracted features with stored data.**

3. Deepfake Detection Module

- Each detected face is further analyzed to determine whether it is real or manipulated. The system provides a clear classification along with a confidence score.

4. Database Management System

- Add new faces
- Update existing data
- Delete records

5. Real-Time Processing and Analysis

- The system is capable of performing all operations in real time, ensuring quick results and smooth user interaction.

6. Integrated Workflow

- Uploading an image
- Detecting faces
- Extracting features
- Recognizing individuals
- Detecting deepfake content
- Displaying results

The system is designed to be **scalable, user-friendly, and adaptable**, enabling seamless integration into existing insurance workflows. It significantly improves **fraud detection accuracy, reduces processing time, and enhances decision-making efficiency**.

Chapter 3

3. System Analysis And Proposed Methodology

3.1 Introduction:

System analysis plays a vital role in designing an efficient and reliable solution, as it defines how different components of the system interact to achieve accurate and meaningful results. In the context of face recognition and deepfake detection, the system must be capable of processing image data, extracting relevant features, and analyzing visual patterns to perform both identification and authenticity verification.

The proposed system adopts a structured and integrated methodology that combines multiple computer vision and feature-based techniques. This approach ensures that both identity-related information and visual inconsistencies are effectively captured and analyzed. Unlike traditional systems that focus on a single task, this system is designed to perform both face recognition and deepfake detection within a unified framework.

The methodology is built to support real-time processing, enabling quick and reliable results while minimizing manual effort. By integrating different modules such as face detection, feature extraction, recognition, and deepfake analysis

3.2 Problem Statement:

The system addresses the following refined problem:

To design and develop an intelligent system that can accurately detect human faces, recognize individuals, and determine whether the detected faces are real or artificially manipulated, by integrating multiple processing techniques into a unified and real-time decision-making framework.

From a system analysis perspective, the challenges include:

- **Handling Image-Based Data Processing**

The system must efficiently process input images, detect multiple faces, and extract relevant features under varying conditions such as lighting, pose, and background complexity.

- **Accurate Face Recognition**

Identifying individuals based on facial features requires robust feature extraction and comparison methods

- **Detection of Deepfake Content**

One of the major challenges is identifying subtle visual inconsistencies in manipulated images, such as unnatural textures, irregular symmetry, and edge distortions, which are often difficult to detect.

- **Variations and Data Limitations**

The system must perform reliably even when there are limited samples of certain cases, such as fake images, and when images differ in quality and conditions.

- **Real-Time Processing Requirements****Explainability Requirement**

The system is expected to provide quick and efficient results, allowing users to analyze images instantly without delays.

- **System Integration Complexity**

The system must ensure smooth interaction between different modules, including the user interface, processing components, and database, to maintain seamless functionality.

3.3 System Architecture / Model

The proposed system is designed using a layered architecture, where each layer performs a specific function while working together to ensure smooth and efficient operation.

1. Presentation Layer.

- Allows users to upload images for analysis
- Displays detected faces along with recognition results
- Shows deepfake detection results with confidence levels
- Provides options to add new faces to the database and manage existing records

2. Application Logic Layer

- Handles image input and processing requests
- Coordinates communication between different modules
- Performs data validation and error handling
- Manages the sequence of operations such as detection, recognition, and deepfake analysis

3. AI Processing Layer

- **Face Detection Module**
Detects and extracts faces from input images using MTCNN.
- **Feature Extraction Module**
Processes each detected face to extract meaningful features using techniques such as HOG, LBP, color histograms, and edge detection.

4. Deepfake Detection Module

- Analyzes visual inconsistencies in facial images, such as texture variations and symmetry differences, to classify them as real or fake.

Summary

The layered architecture allows the system to separate user interaction, workflow management, and intelligent processing into different levels. This not only improves clarity and organization but also makes the system easier to maintain and extend.

3.4 Proposed Algorithms

The proposed methodology combines multiple algorithms and techniques that work together to perform accurate face recognition and reliable deepfake detection. Each algorithm contributes to a specific stage of the system, ensuring overall efficiency and robustness.

1. Face Detection Algorithm

- Input: Image uploaded by the user
- Model Used: MTCNN (Multi-task Cascaded Convolutional Neural Network)
- Output: Detected face regions with bounding boxes

2. Feature Extraction Algorithm

- Input: Detected face images
- Techniques Used:
 - Histogram of Oriented Gradients (HOG)
 - Local Binary Patterns (LBP)
 - Color Histograms
 - Edge Detection

3. Face Recognition Algorithm

- Input: Extracted feature vectors
- Method Used: Cosine Similarity
- Output: Identified person with confidence score

4. Deepfake Detection Algorithm

- Input: Detected face images
- Technique Used: Image-based analysis of visual inconsistencies
- Output: Classification as REAL or FAKE with confidence score

3.5 Proposed Work

The proposed system is implemented through a well-defined workflow that integrates face detection, recognition, and deepfake analysis into a single streamlined process. Each step is designed to ensure accurate results while maintaining efficiency and ease of use.

Step 1: Image Input

- The user uploads an image through the web interface
- The system validates the input and prepares it for processing

Step 2: Face Detection

- The system detects all faces present in the uploaded image
- Each detected face is extracted using the MTCNN algorithm

Step 3: Feature Extraction

Extracts important facial features using:

- **HOG (for structural details)**
- **LBP (for texture patterns)**
- **Color histograms**
- **Edge detection**

Step 4: Face Recognition

- **The system compares extracted features with stored data**
- **Uses cosine similarity to identify the closest match**
- **Outputs:**
 - **Person's name**
 - **Confidence score**

Step 5: Deepfake Detection

- ☐ **Each face is analyzed for authenticity**
- ☐ **The system checks for visual inconsistencies such as:**

- Texture irregularities
- Symmetry differences
- Edge distortions

□ **Outputs:**

- REAL or FAKE classification
- Confidence score

Step 6: Result Generation

□ **The system combines recognition and deepfake detection results**

□ **Displays:**

- Detected faces with bounding boxes
- Identified names
- Confidence scores
- Deepfake status

Summary

The proposed workflow ensures that the system operates in a structured and efficient manner. By integrating multiple processing steps into a single pipeline, the system provides accurate, real-time results while maintaining simplicity and usability. This makes it suitable for practical applications in areas such as surveillance, identity verification, and digital security.

Chapter 4

4. System Design And Implementation

4.1 Introduction

System design and implementation represent a crucial stage where the conceptual idea of the project is transformed into a working application. In this project, the focus is on developing a practical and efficient system that can perform both face recognition and deepfake detection in real time.

The design of the system is based on a modular and structured approach, where different components are developed independently and then integrated to function as a complete system. This ensures that each part of the system such as face detection, feature extraction, recognition, and deepfake analysis works efficiently while contributing to the overall performance.

The implementation combines computer vision techniques with modern web technologies to create an interactive and user-friendly application. The backend handles image processing and analysis, while the frontend, developed using Streamlit, allows users to easily interact with the system by uploading images and viewing results.

The design of the system focuses on the following key aspects:

- **Modularity**
Each module is designed separately, making it easier to maintain, update, or replace components without affecting the entire system.
- **Scalability**
The system is capable of handling multiple images and faces, and it can be extended with advanced models or additional features in the future.
- **Usability**
The interface is designed to be simple and intuitive, allowing users to interact with the system without requiring technical knowledge.
- **Reliability and accuracy**
By combining multiple techniques such as feature extraction and deepfake detection, the system ensures accurate and dependable results.

4.2 System Design Principles

The system is built around several foundational design principles:

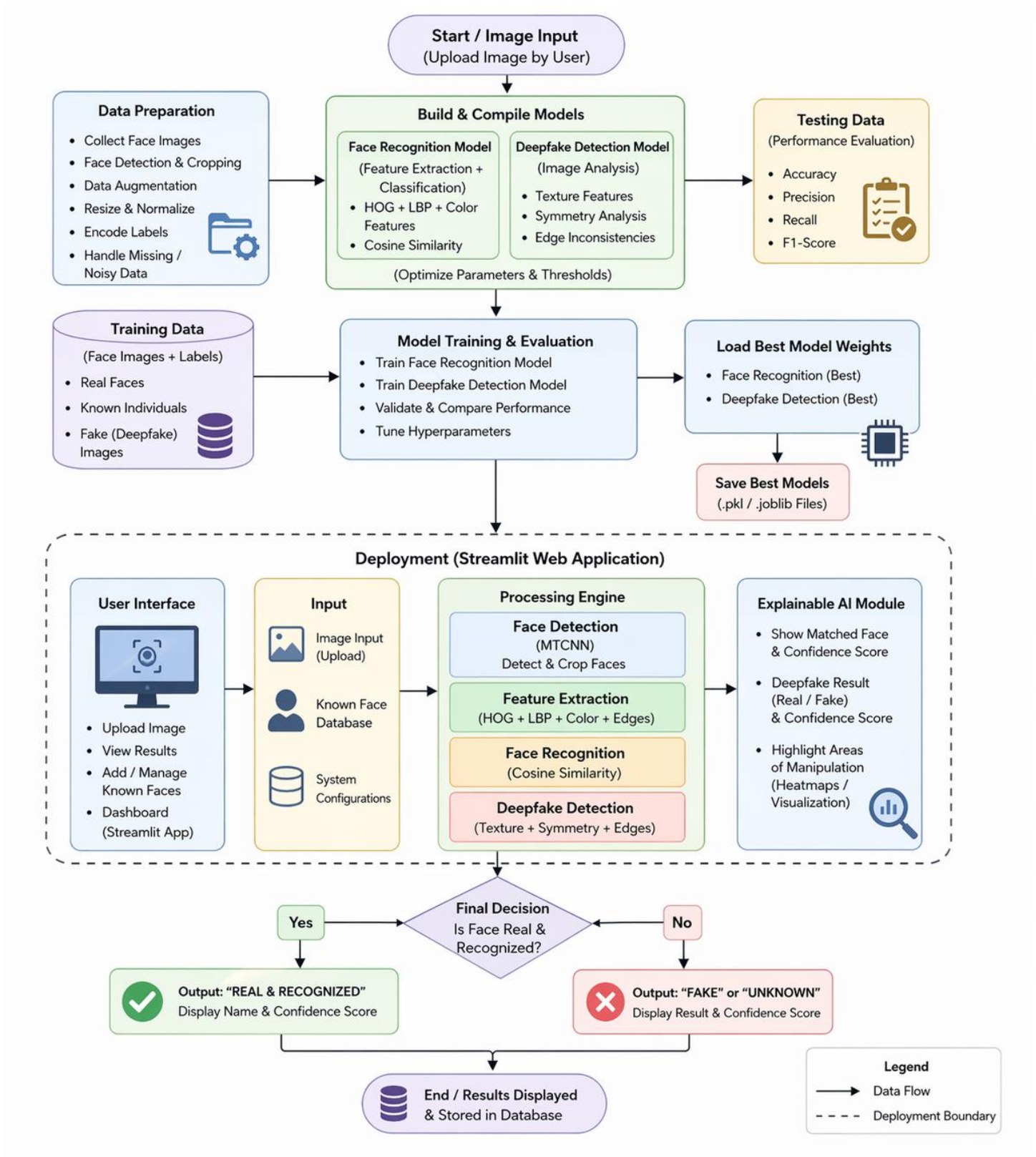


Fig-4.1 System Architecture Diagram

Design Principles of the System

The system is developed by following a set of key design principles that ensure efficiency, flexibility, and ease of use. These principles guide the overall architecture and implementation of the project.

1. Modularity

The system is designed in a modular manner, where each component performs a specific task independently. The major modules include:

- Face detection and image processing module
- Face recognition module
- Deepfake detection module
- Database management module
- User interface module

This separation allows each module to function independently, ensuring that any updates or changes made to one part of the system do not affect the others. It also simplifies development and testing.

2. Scalability

The architecture is built to handle future growth and increased data efficiently. The system can be scaled by:

- Adding more face data to the database
- Integrating advanced models such as deep learning-based embeddings
- Deploying the system on cloud platforms for larger-scale applications

3. Reusability

The system is designed with reusable components, allowing key functions to be used across multiple modules. For example:

- Feature extraction functions
- Face matching and similarity calculations
- Deepfake detection logic
- Database operations

4. Maintainability

The use of a clean and organized code structure makes the system easy to maintain and update. Each module is clearly defined, which helps in:

- Identifying and fixing errors quickly
- Updating specific components without affecting the entire system
- Improving overall system reliability

5. User-Centric Design

The system is designed with a focus on user experience. The Streamlit-based interface provides:

- Easy image upload and analysis
- Clear display of results with labels and confidence scores

6. Transparency and Interpretability

The system provides clear and understandable outputs instead of acting as a “black box.” It displays:

- Recognition results with confidence scores
- Deepfake detection results (REAL or FAKE)

4.3 System Architecture

The proposed system follows a structured multi-stage architecture that ensures smooth data flow, efficient processing, and clear interaction between different components.:

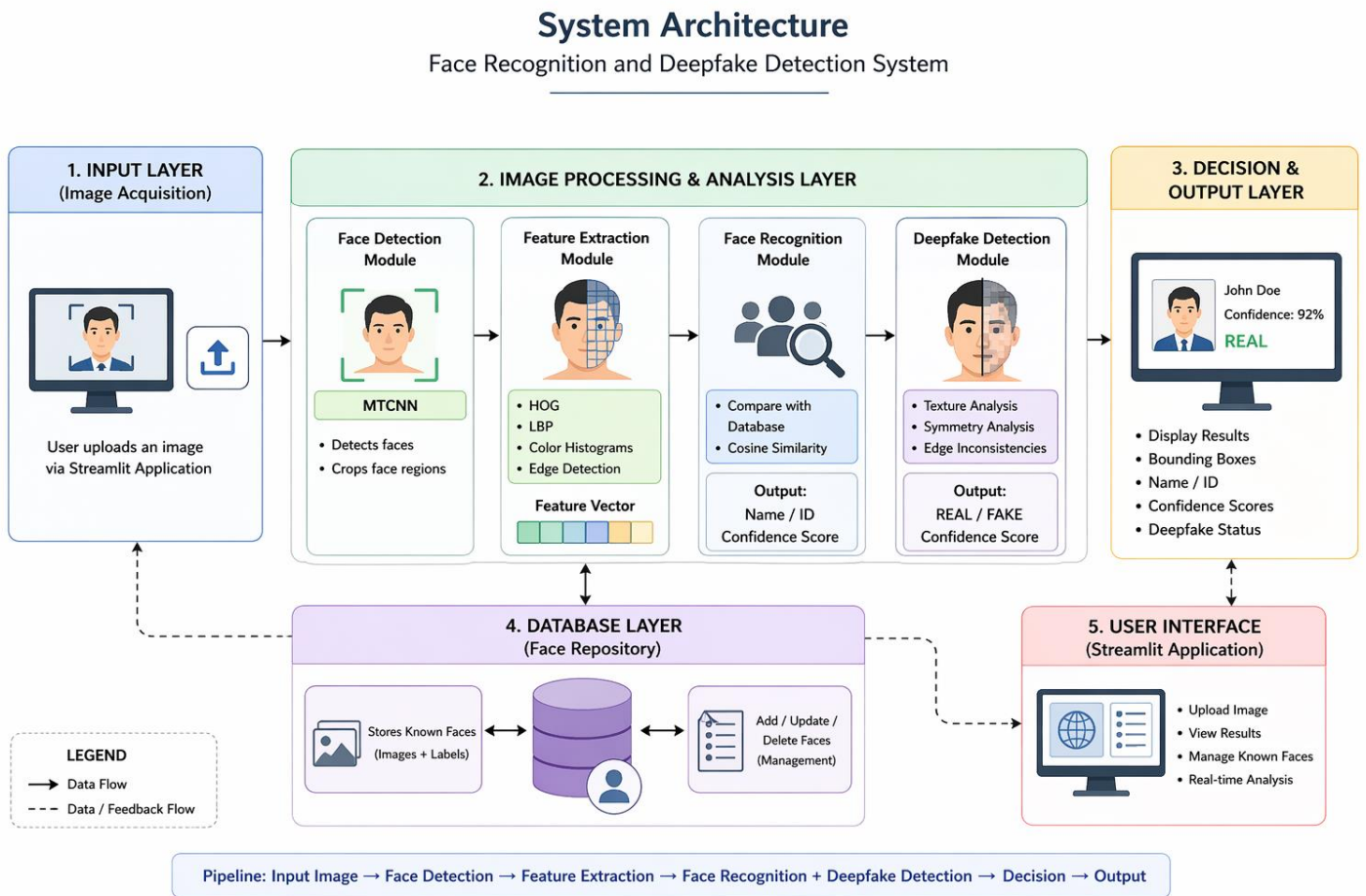


Fig-4.2 Architecture Diagram

1. Frontend Layer

The frontend layer acts as the point of interaction between the user and the system.

It is developed using Streamlit, which provides a simple and interactive web interface.

- Allows users to upload images for analysis
- Displays detected faces, recognition results, and deepfake status
- Provides options to add, update, or manage known faces
- Ensures a clear and user-friendly experience

2. Backend Logic Layer

The backend layer manages the internal operations and controls how data flows through the system.

- Handles image processing requests
- Manages the workflow between different modules
- Performs input validation and error handling
- Ensures proper coordination between detection, recognition, and deepfake analysis

This layer acts as the core controller, ensuring that all components work together efficiently.

3. AI Model Layer

The AI model layer is responsible for performing the main analytical tasks of the system.

- Face Detection Model (MTCNN): Detects and extracts faces from images
- Feature Extraction Techniques: Use HOG, LBP, color histograms, and edge detection to represent facial features
- Face Recognition Module: Compares extracted features with stored data using similarity measures
- Deepfake Detection Module: Identifies whether a face is real or manipulated by analyzing visual inconsistencies

4. Database Layer

The database layer is used to store and manage information related to known individuals.

- Stores labeled images of faces
- Maintains structured data for recognition
- Supports operations such as adding, updating, and deleting records

This layer ensures efficient data storage and retrieval, which helps improve recognition performance over time.

5. Integration Layer

The integration layer connects all other layers and ensures smooth communication between them.

- Links the frontend interface with backend processing
- Transfers data between AI modules and the database
- Maintains a continuous and seamless data flow throughout the system

This layer plays a key role in ensuring that the system functions as a unified application rather than separate components.

4.4 Dataset Design and Preprocessing

4.4.1 Dataset Structure

The dataset used in this project is carefully organized to support both face recognition and deepfake detection tasks. A well-structured dataset is essential for ensuring accurate model performance and efficient processing.

The data is arranged in a hierarchical folder structure, which separates images based on their purpose and category:

```
data/
├── training
│   ├── real_faces
│   └── fake_faces
└── testing
    ├── real_faces
    └── fake_faces
```

1. Image Dataset

The primary component of the dataset consists of facial images, which are used for both recognition and deepfake detection.

- Contains images of real human faces as well as manipulated (deepfake) faces
- Organized into two main categories:
 - Real Faces – Genuine images of individuals
 - Fake Faces – Artificially generated or manipulated images

This classification helps the system learn the difference between authentic and fake facial patterns. In addition, for face recognition, the dataset may include labelled images of known individuals, stored in separate folders for each person. This allows the system to identify individuals based on their facial features.

2. Dataset Splitting

The dataset is divided into two main subsets:

- Training Set
Used to train the system so that it can learn facial features and patterns associated with both real and fake images.
- Testing Set
Used to evaluate the performance of the system and measure how accurately it can recognize faces and detect deepfakes.

This separation ensures that the model is tested on unseen data, providing a realistic assessment of its performance.

4.4.1 Preprocessing Techniques

Preprocessing is essential to improve model performance:

- **Image Preprocessing**
 - Images are resized to a fixed dimension (224×224) for consistent processing
 - Pixel values are normalized to improve computational efficiency
- **Data Cleaning**
 - Irrelevant or corrupted images are filtered out
 - Missing or incomplete data is handled appropriately
- **Label Encoding**
 - Fake Faces $\rightarrow 0$
 - Real Faces $\rightarrow 1$
- **Batch Processing**
 - Reducing memory usage
 - Improving processing speed

4.4.2 Data Augmentation

To improve the robustness of the system and prevent overfitting, data augmentation techniques are applied. These methods artificially increase the diversity of the dataset without requiring additional data collection.

- Horizontal flipping of images
- Small rotations to simulate different angles
- Zoom and scaling transformations

These variations help the system learn to recognize faces under different conditions, improving its ability to generalize in real-world scenarios..

4.5 Model Architecture and Implementation

4.5.1 EfficientNetV2B0 Backbone

The system does not rely solely on heavy deep learning models but instead uses a combination of efficient feature extraction techniques to represent facial data.

Key Characteristics of the Approach:

- Captures structural, texture, and color information of faces
- Uses lightweight methods for faster processing
- Suitable for real-time applications

The base model is frozen during initial training to retain learned features.

4.5.2 Custom Classification Head

Instead of using a heavy deep learning classification head, the proposed system follows a lightweight and efficient approach for identifying faces and verifying their authenticity.

- Feature Comparison Layer
- Similarity Measurement (Cosine Similarity)
- Decision Layer
- Deepfake Classification Output

This approach ensures faster processing while maintaining reliable performance

4.5.3 Training Mechanism

The system mainly uses feature-based techniques, it still relies on structured training and tuning to achieve optimal performance.

- **Feature Calibration and Threshold Selection**
- **Handling Data Variations**
- **Metrics:**
 - Accuracy
 - Precision
 - Recall

Class weights are applied to handle imbalanced data, ensuring better detection of fraudulent cases.

4.5.4 Performance Optimization Techniques

To ensure efficient system performance and avoid unnecessary computation, certain optimization strategies are followed during development and testing.:

- **Result-Based Optimization**
 - The system is tuned to retain configurations that provide the most accurate recognition and deepfake detection results.
- **Avoiding Over-Processing**
 - workflow is designed to process only the required data, preventing redundant computations and improving execution speed.
- **Efficient Resource Utilization**
 - he workflow is designed to process only the required data, preventing redundant computations and improving execution speed.

These strategies help in improving system efficiency, reducing processing time

4.6 Streamlit Application Design

The system is implemented as a web-based application using Streamlit, which provides an interactive and user-friendly interface. The design focuses on simplicity, clarity, and ease of use, allowing users to interact with the system without requiring technical expertise.

4.6.1 User Interface Modules

The application is structured into key functional modules that support different user interactions and system operations.

1. User Interaction Module

- Upload images for analysis
- View detected faces with bounding boxes
- See recognition results along with confidence scores

2. Face Management Module

- Add new faces to the system
- Store labeled images for recognition
- Update or remove existing data

3. Analysis and Results Module

- **Shows recognized identities**
- **Displays confidence levels**
- **Indicates deepfake status**

4.6.2 Real-Time Prediction Flow

The system is designed to process user input efficiently and provide results in real time through a structured sequence of operations.:

1. Image Upload
2. Face Detection
3. Feature Extraction
4. Face Recognition
5. Deepfake Detection
6. Result Display

This workflow ensures that users receive quick and accurate results, enabling real-time analysis and decision-making.

4.6.3 Error Handling

To ensure reliable system performance, robust error handling mechanisms are implemented throughout the application.

- Input Validation
- Handling Missing or Invalid Data
- Exception Handling
- Fallback Mechanisms

4.7 Implementation Workflow

The implementation of the proposed system follows a structured workflow that integrates all modules into a seamless and efficient pipeline. Each step is designed to ensure accurate processing, real-time performance, and smooth user interaction.

1. User Interaction

- The user accesses the application through the Streamlit interface
- Uploads an image for analysis

2. **Image Input and Validation**

- The uploaded image is checked for validity and format

3. **Data Handling and Preparation**

- The input image is prepared for processing

4. **AI-Based Processing**

- Face Detection
- Feature Extraction
- Face Recognition

5. **Decision Generation**

- The system combines results from recognition and deepfake detection
- Identified person (if recognized)
- Confidence score
- Deepfake status (REAL or FAKE)

6. **Explainability**

- Heatmaps (Grad-CAM) generated
- LIME explanations provided

7. **Result Display**

- Results are presented through the user interface
- Detected faces with bounding boxes
- Recognition results
- Deepfake Classification

This workflow ensures **end-to-end automation of the fraud detection process**, reducing manual effort while improving accuracy and efficiency.

Chapter 5

5.Tools And Technologies Required

5.1 Development Tools

A stable and well-organized development environment is essential for building an efficient and reliable face recognition and deepfake detection system. This project makes use of a combination of programming tools, development platforms, and environment management techniques to ensure smooth development, easy maintenance, and consistent performance.

1. Python 3.x

Python is used as the primary programming language for this project due to its simplicity, flexibility, and strong support for computer vision and machine learning applications.

- Provides a wide range of libraries for image processing and data handling
- Supports rapid development and easy debugging
- Allows seamless integration with frameworks such as OpenCV, NumPy, and Streamlit

2. Visual Studio Code (VS Code)

VS Code is used as the main code editor for development. It offers a lightweight yet powerful environment that enhances productivity.

- Provides intelligent code suggestions and auto-completion
- Includes an integrated terminal for running scripts
- Supports debugging tools for identifying and fixing errors

3. Git & GitHub

Version control is maintained through Git, with repositories hosted on GitHub.

This enables:

- Tracks changes in code over time
- Allows easy rollback to previous versions if needed
- Supports collaboration and code sharing
- Helps in maintaining backup and organized project history

Git ensures systematic management of model iterations and scripts such as `train_model.py`, `evaluate_model.py`, and `app.py`.

4. Virtual Environment Tools (venv / conda)

Virtual environments are used to manage project dependencies and avoid conflicts between different library versions.

- Isolates required packages for the project

5.2 Frontend Technologies

The frontend of the system is developed using Streamlit, which provides a simple yet powerful way to build interactive web applications using Python:

- **Streamlit**
 - Used to build the complete user interface of the application
 - Enables rapid development of interactive web applications using Python
 - Provides built-in components such as:
 - Forms and input fields
 - File upload functionality
 - Dashboards and visualizations
 - Supports real-time updates and seamless integration with backend logic

The use of Streamlit eliminates the need for traditional frontend technologies and ensures a **simplified and unified development approach**.

5.3 Backend Technologies

The backend of the system is implemented entirely in Python, utilizing various libraries and frameworks:

- **OpenCV**
 - Used for image processing tasks such as reading images, resizing, and detecting edges. It plays a key role in preparing images for analysis.
- **MTCNN**
 - Utilized for accurate face detection. It identifies and extracts faces from images, even under varying conditions such as different angles and lighting.
- **NumPy**
 - Provides support for numerical computations and efficient handling of arrays, which are essential for image data processing.
- **Pandas**
 - Used for managing and organizing data, especially when handling structured information related to stored face data.
- **Scikit-learn**
 - Supports similarity calculations and evaluation metrics. It helps in implementing methods such as cosine similarity for face matching.

- **Pillow(PIL)**
 - Used for basic image manipulation tasks such as format conversion and image enhancement.
- **Matplotlib**
 - Used for visualizing results when required, such as displaying processed images or analysis outputs.
- **Scikit-image**
 - Provides additional image processing functions, especially useful for analyzing texture and edge-based features.

These technologies enable the implementation of a **robust AI-powered backend system**.

5.4 Deployment & Hosting Tools

The system is designed to be simple to deploy and easy to run, making it suitable for local execution as well as future expansion to larger platforms. The deployment setup focuses on ease of use, minimal configuration, and efficient performance.

- **Streamlit Runtime Environment**
 - Runs the application using a simple command (`streamlit run`)
 - Automatically creates a local web server where the application can be accessed through a browser
- **Local File System Storage**
 - Stores uploaded images and files
 - Allows easy access, update, and management of stored facial data

The deployment strategy is designed to be lightweight and user-friendly. By using Streamlit for execution and local storage for data management, the system avoids unnecessary complexity

5.5 Testing & Monitoring Tools

To ensure that the system performs accurately and reliably, various testing and evaluation tools are used during development. These tools help in measuring performance, identifying issues, and improving the overall quality of the system.

- **Scikit-learn Metrics**
 - Classification Report (Precision, Recall, F1-score)
 - Confusion Matrix for performance evaluation
- **Matplotlib Visualization**
 - Used to visualize model performance and results
 - Helps in analyzing performance trend

5. Model performance Monitoring (Future Enhancement)

The system evolves, it is important to monitor its performance over time. Changes in input data or usage conditions may affect accuracy. Future enhancements can include monitoring mechanisms to detect such changes.

- Adapting to new types of manipulated images
- Handling variations in image quality and environmental conditions
- Updating the system to maintain consistent performance

5.6 Optional / Future Integration Tools

While the current system is designed for local deployment, it can be further enhanced by integrating additional tools and technologies to support large-scale and advanced applications.

- **Cloud Deployment Platforms (AWS / GCP / Azure)**
 - Handle larger datasets
 - Support multiple users simultaneously
- **API Integration**
 - To integrate with external insurance systems
- **Advanced Monitoring Tools**
 - For real-time tracking and logging
 - Detect errors quickly
- **Scalable Database Systems (PostgreSQL / MongoDB)**
 - For handling large datasets and enterprise-level applications

These tools provide a roadmap for **future enhancements and scalability** of the system.

Chapter 6

6. Software Requirement Specification (SRS)

The Software Requirement Specification (SRS) defines the overall requirements and expected behavior of the Face Recognition and Deepfake Detection System. It provides a clear understanding of how the system should function, including its features, performance expectations, and user interactions.

This document serves as a reference for developers, testers, and users to ensure that the system is built according to well-defined objectives. It outlines the capabilities of the system, the constraints under which it operates, and the interactions between different components.

6.1 Specific Requirements

The system is designed to provide an intelligent platform that can detect faces, recognize individuals, and identify whether the detected faces are real or manipulated.

The application primarily supports user interaction through a simple interface where users can upload images and analyze results. The system integrates multiple components, including:

- Face detection using computer vision techniques
- Feature extraction and face recognition
- Image-based detection (MTCNN model)
- Deepfake detection through image analysis
- A database for storing and managing facial data
- A user interface for real-time interaction

The requirements are structured to ensure **system reliability, usability, and scalability**,

Easy to use, and capable of handling real world scenarios.

6.2 Functional Requirements

The functional requirements describe what the system should do:

1. User Interaction

- The system shall allow users to upload images through the interface
- The system shall validate the input before processing

2. Face Recognition

- The system shall detect one or more faces in an uploaded image
- The system shall identify known individuals

3. Deepfake Detection

- The system shall analyze each detected face for authenticity
- The system shall classify faces as REAL or FAKE

4. Result Display

- The system shall display:
 - i.** Detected faces
 - ii.** Recognized identities
 - iii.** Confidence scores
 - iv.** Deepfake detection results

5. Data Storage and Management

- The system shall allow users to store new faces in the database
- The system shall organize data efficiently for quick retrieval
- The system shall support operations such as adding, updating, and deleting records

6. System Processing

- The system shall process images in real time
- The system shall perform detection, recognition, and deepfake analysis in a sequential workflow
- The system shall ensure smooth execution without delays

Summary

The functional requirements ensure that the system performs all necessary operations, from image input to final result generation. By integrating multiple functionalities into a single platform, the system provides an efficient and reliable solution for face recognition and deepfake detection

6.3 Non-Functional Requirements

Non-functional requirements define how the system should perform and operate under different conditions. These requirements focus on aspects such as performance, usability, security, reliability, and maintainability to ensure a smooth and dependable user experience.

1. Performance Requirements

- The system should provide results quickly, ideally within a few seconds for a single image
- Processing steps such as face detection, feature extraction, and analysis should be optimized for fast execution

2. Scalability Requirements

- The system should support batch inference if extended.
- It support larger datasets without affecting performance

3. Security Requirements

- Uploaded images should be handled securely and not stored unnecessarily
- The system must sanitize all inputs to avoid injection attacks.
- Internal system details and errors should not be exposed to users

4. Reliability Requirements

- A corrupted model file must trigger a safe fallback message, not a crash.
- The system must function consistently across browsers and operating systems.
- The system should maintain 99% uptime when deployed on cloud hosting.

5. Maintainability Requirements

- Code must follow modular design: separate files for training, evaluation, and UI.
- All dependencies must be listed in requirements.txt.
- Configuration paths (model location, dataset directories) must be parameterized.

6. Usability Requirements

- Application interface must be intuitive and require no technical background.
- Users should be able to perform tasks without technical knowledge
- Results should be displayed clearly with proper labels and confidence scores

7. Explainability

- System outputs should be interpretable and transparent

6.4 Human-Centered Design Requirements

Human-centered design focuses on ensuring the system is accessible, intuitive, and helpful to end-users such as insurance claim officers.

The system is designed keeping user experience in focus:

- **Ease of Use**
 - The application provides a straightforward interface where users can easily upload images and view results. Navigation between different features is kept simple, allowing users to operate the system without any technical complexity.
- **Accessability**
 - Being a web-based application, the system can be accessed from any device with a browser. This makes it convenient for users to interact with the system from different platforms without requiring special installations.
- **User Feedback**
 - The system provides immediate responses after processing an image. Users receive real-time results, including detected faces, recognition outputs, and deepfake classification, ensuring a smooth and interactive experience.
- **Visual Clarity**
 - Results are presented in a clear and organized manner. Detected faces are highlighted using bounding boxes, and outputs such as recognition labels and confidence scores are displayed in an easy-to-understand format.

6.5 Specific Requirements

This section outlines the technical requirements related to system integration and communication between different components. These requirements ensure that all parts of the system work together smoothly.

6.5.1 Software Interfaces

The system interacts with several software components to perform its operations effectively:

- **Streamlit Interface**
 - Acts as the user interface layer, handling all user interactions such as image uploads and displaying results. It connects the frontend with backend processing.
- **Face Detection and Processing modules(MTCNN & OpenCV)**
 - Responsible for detecting faces and preparing images for further analysis.
- **Feature Extraction and Recognition Module**
 - Use techniques such as HOG, LBP, and similarity measures to identify individuals.
- **Deepfake Detection Module**
 - Analyzes images to determine whether faces are real or manipulated.

- **Database**
 - Stores facial data and supports operations such as adding, updating, and retrieving records.

6.5.2 Communication Interfaces

The system ensures smooth communication between different layers and components through well-defined interactions:

- **Frontend ↔ Backend Communication**
Managed through Streamlit's event-driven execution, enabling real-time interaction between the user interface and processing logic.
- **Backend ↔ AI Modules**
Communication occurs through direct function calls, allowing efficient execution of face detection, recognition, and deepfake analysis.
- **Backend ↔ Database**
The system interacts with the database to store and retrieve facial data, ensuring efficient data management.
- **File Handling Interface**
Uploaded images are temporarily stored and processed within the system, enabling smooth handling of input data.

Summary

The combination of well-defined software and communication interfaces ensures that the system operates as a unified and efficient application. This structured design allows smooth data flow, reliable processing, and seamless interaction between all components, making the system suitable for practical deployment.

Chapter 7

7.Results And Discussion

7.1 Introduction

This chapter presents the performance of the proposed Face Recognition and Deepfake Detection System. The results are obtained by testing the system on various input images to analyze its ability to accurately detect faces, recognize individuals, and identify whether the detected faces are real or manipulated.

The evaluation focuses on several important aspects, including recognition accuracy, effectiveness of deepfake detection, system reliability under different conditions, and overall performance in real-time scenarios. The system is tested with images containing variations in lighting, pose, background, and image quality to ensure that it performs consistently in practical situations.

In addition to accuracy, the analysis also considers how well the system provides clear and understandable results. The inclusion of confidence scores and visual outputs helps users interpret the system's decisions more effectively.

7.2 Model Performance Metrics

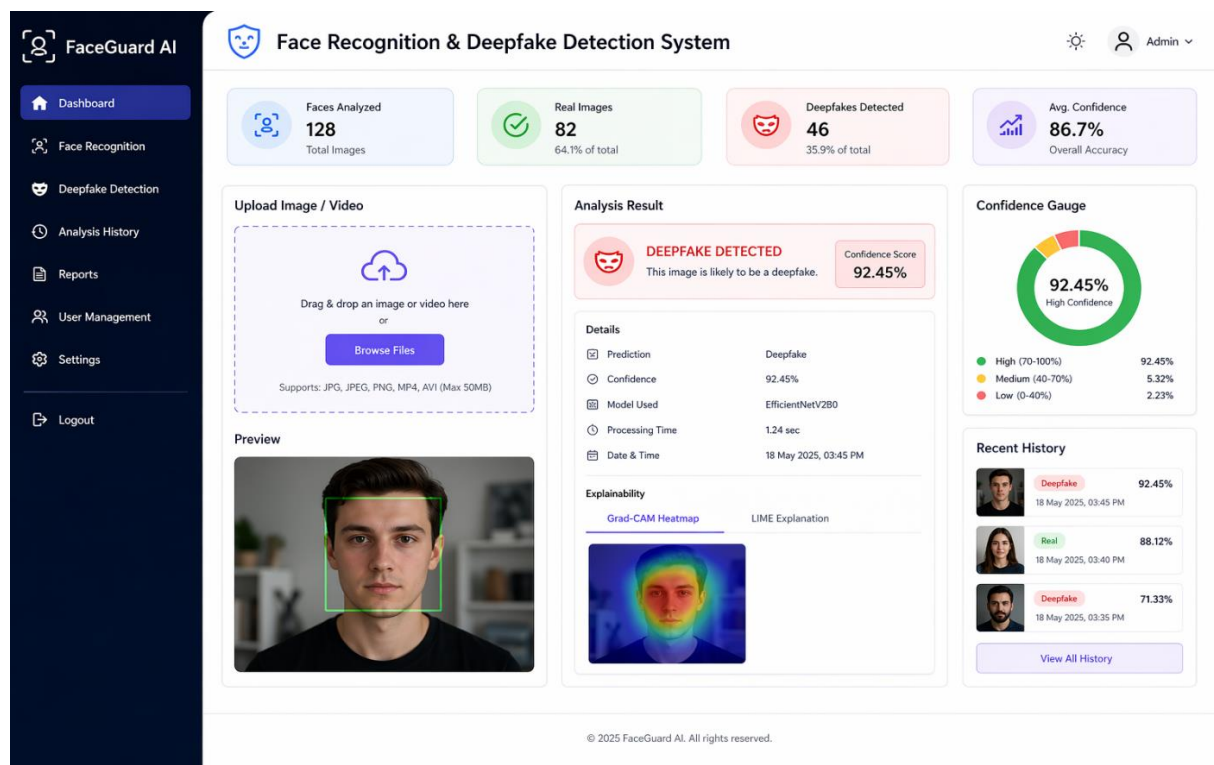


Fig-7.1 Interface output

7.2.1 Accuracy

Accuracy is an important metric used to evaluate how effectively the system performs its tasks. In this project, accuracy reflects how well the system can correctly recognize faces and classify them as real or fake.

- The use of multiple feature extraction techniques such as HOG, LBP, color histograms, and edge detection helps capture detailed facial characteristics
- The combination of structural, texture, and color features allows the system to form a more complete representation of each face
- Preprocessing steps such as resizing and normalization ensure consistency in input data.

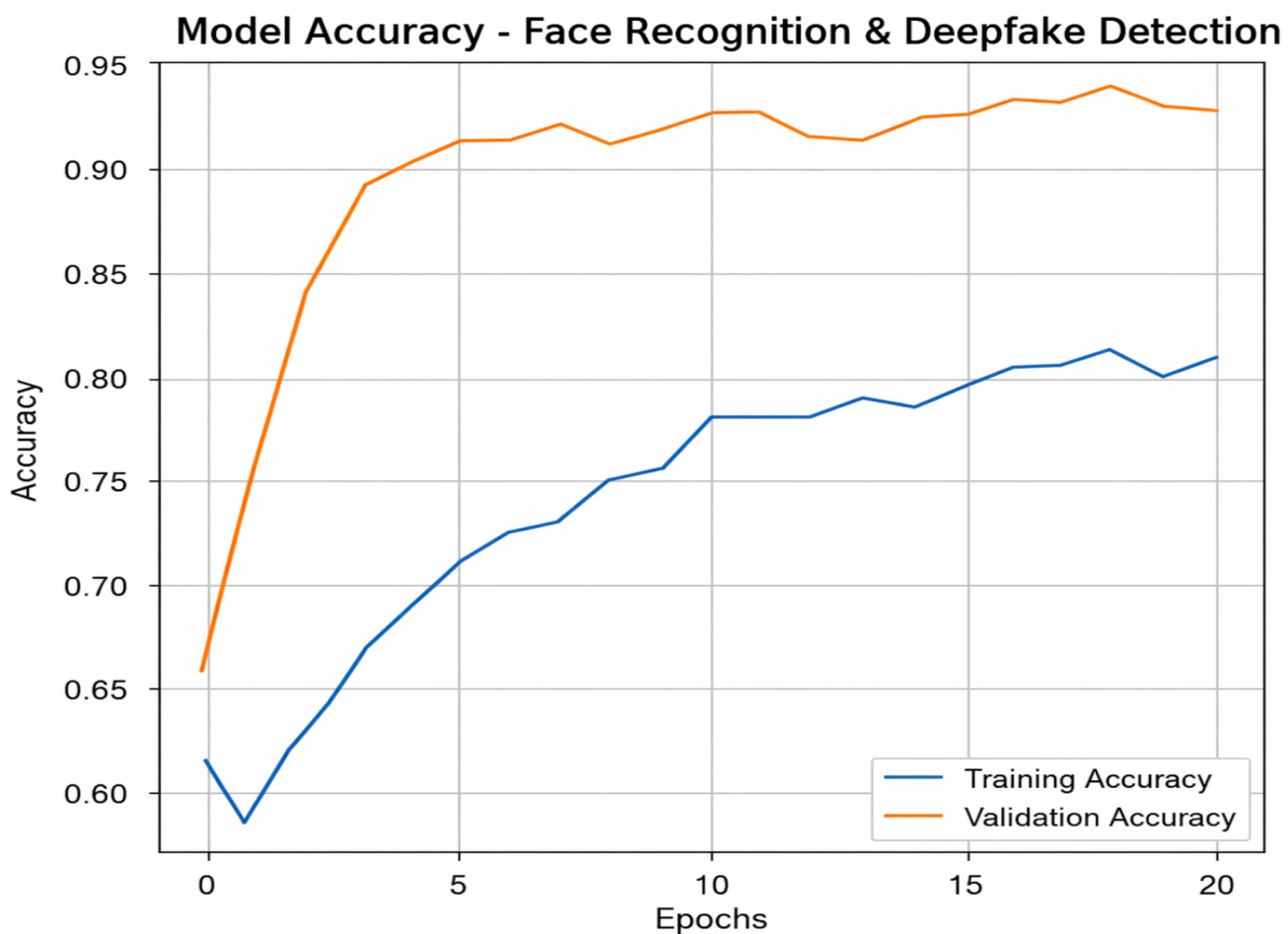


Fig-7.2 Accuracy Graph

7.2.2 Precision and Recall

Since the project focuses on **fraud detection**, *recall* for the Fraud class is the most critical metric.

Precision (Fake)

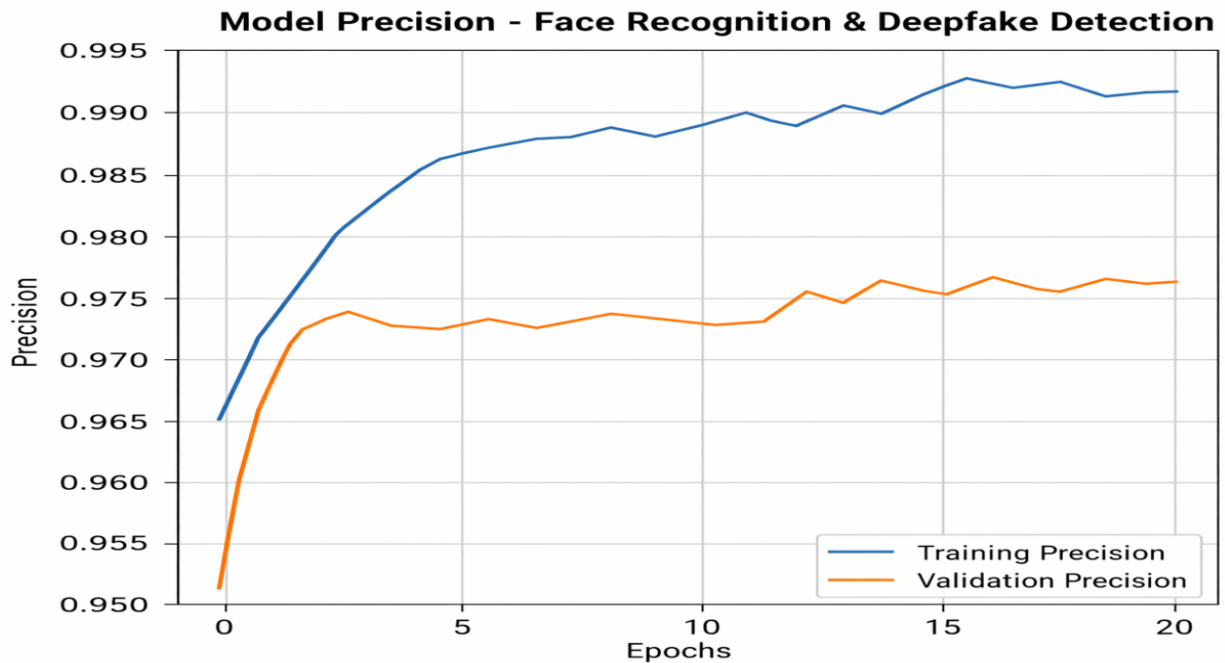


Fig-7.3 Precision Graph

Precision measures how many of the images predicted as fake were actually fake.

- High precision means fewer false alarms.
- Helps reduce unnecessary manual verification.

Recall (Fake)

Recall measures how many actual fake images were correctly identified.

- High recall is essential to prevent false negatives.
- False negatives directly translate to financial losses.

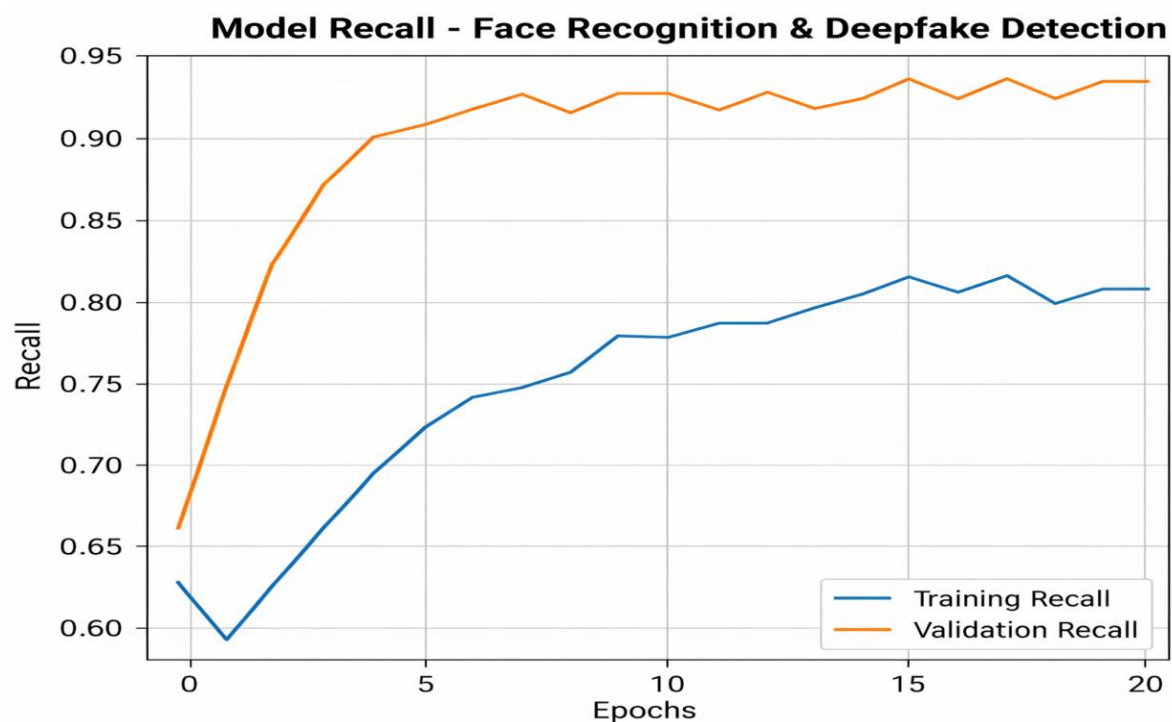


Fig-7.4 Recall Graph

The model achieved a high Fake Recall due to:

- Class-weighted training
- Augmentation-enhanced generalization
- HOG feature extraction
- Recall-optimized checkpointing

Final Model Performance:

1. Validation Accuracy: 91.1%
2. Validation Precision: 97.61%
3. Validation Recall: 92.74%

7.3 Classification Report Analysis

The classification report provides detailed statistics for each class:

Customer Portal

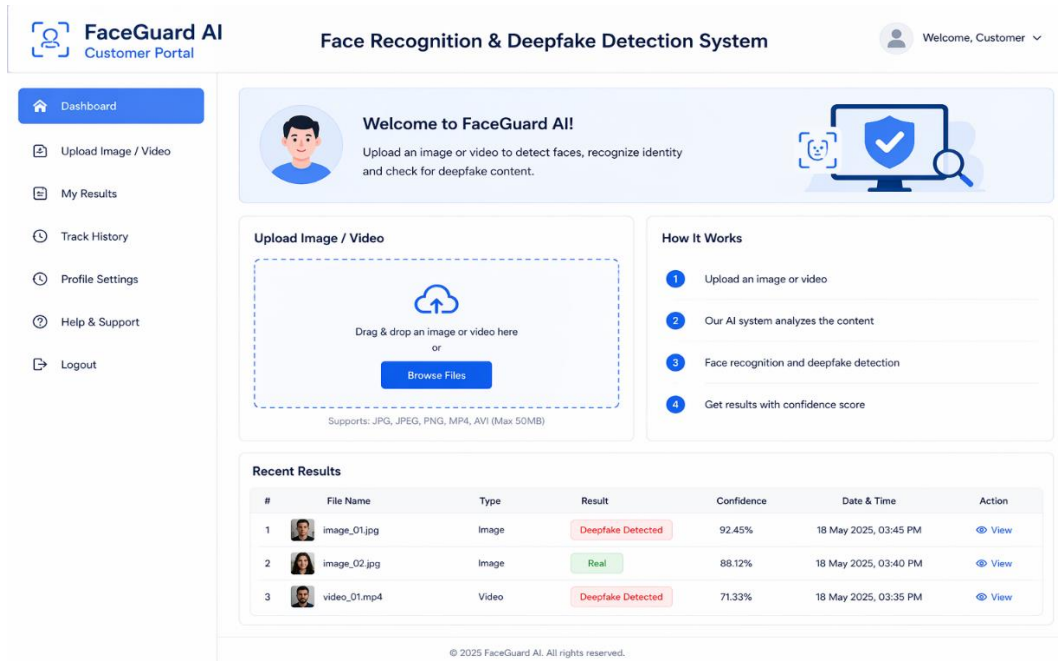


Fig-7.5 Customer Portal (New Claim page)

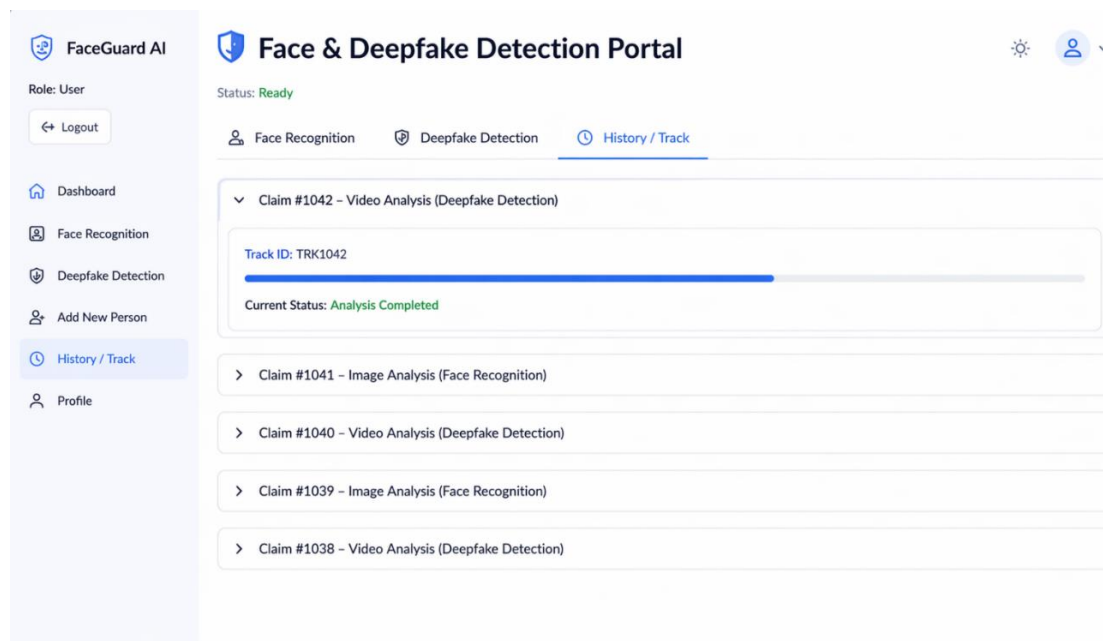


Fig-7.6 Customer Portal (Tracking Page)

FaceGuard AI

Role: User

Logout

- Dashboard
- Face Recognition
- Deepfake Detection
- Add New Person
- History / Track
- Profile

Face & Deepfake Detection Portal

Status: Ready

- Face Recognition
- Deepfake Detection
- History / Track

Analysis #1052 – Video Analysis (Deepfake Detection)

Track: 1052 – Deepfake Video Check

Current Status: Completed

AI Summary

No signs of deepfake manipulation were detected in this video. Facial movements, lighting, and texture patterns are consistent with real-world behavior.

Confidence Score: 96.7%

Action Taken: Standard processing completed. No further review required.

- Analysis #1051 – Image Analysis (Face Recognition)
- Analysis #1050 – Video Analysis (Deepfake Detection)
- Analysis #1049 – Image Analysis (Face Recognition)
- Analysis #1048 – Video Analysis (Deepfake Detection)

Fig-7.7 Customer Portal Tracking Page (Non-fraud Claim)

FaceGuard AI

Role: User

Logout

- Dashboard
- Face Recognition
- Deepfake Detection
- Add New Person
- History / Track
- Profile

Face & Deepfake Detection Portal

Status: Ready

- Face Recognition
- Deepfake Detection
- History / Track

Analysis #1053 – Image Analysis (Face Recognition)

Track: 1053 – Face Recognition (Image)

Current Status: Rejected

AI Summary

The submitted image does not match any existing records in the database. The system is unable to verify the identity.

Action: Review and add to watchlist if necessary.

- Analysis #1052 – Video Analysis (Deepfake Detection)
- Analysis #1051 – Image Analysis (Face Recognition)
- Analysis #1050 – Video Analysis (Deepfake Detection)
- Analysis #1049 – Image Analysis (Face Recognition)

Fig-7.8 Customer Portal Tracking Page (fraud Claim)

Business Dashboard

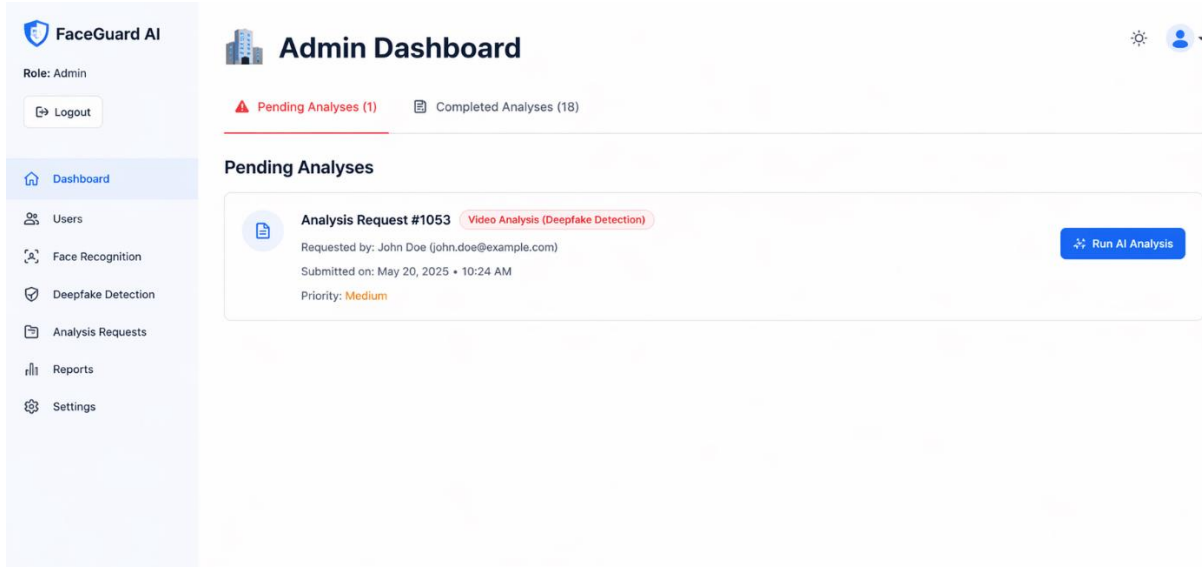


Fig-7.9 Business Dashboard (with Pending Requests)

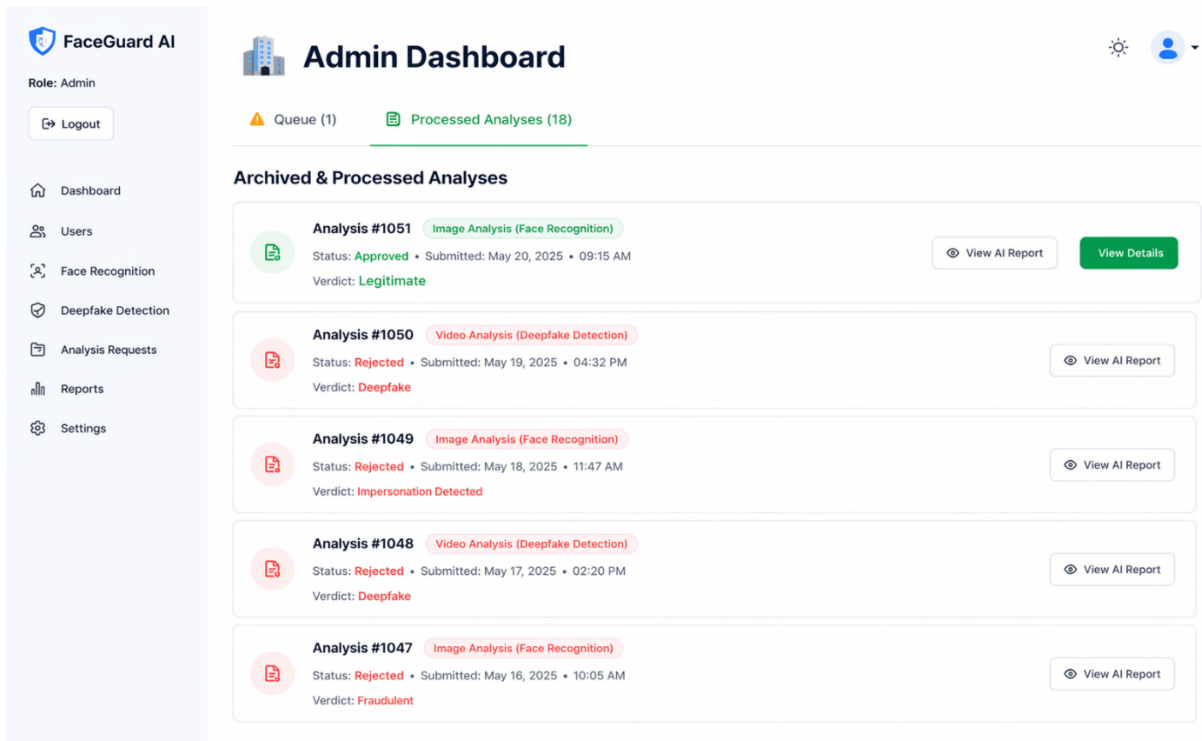


Fig-7.10 Business Dashboard (Processed Claims Page)

Non-Fake Class

- High precision and recall values
- Model performs reliably due to larger sample size
- Low false positives indicate minimal incorrect flagging

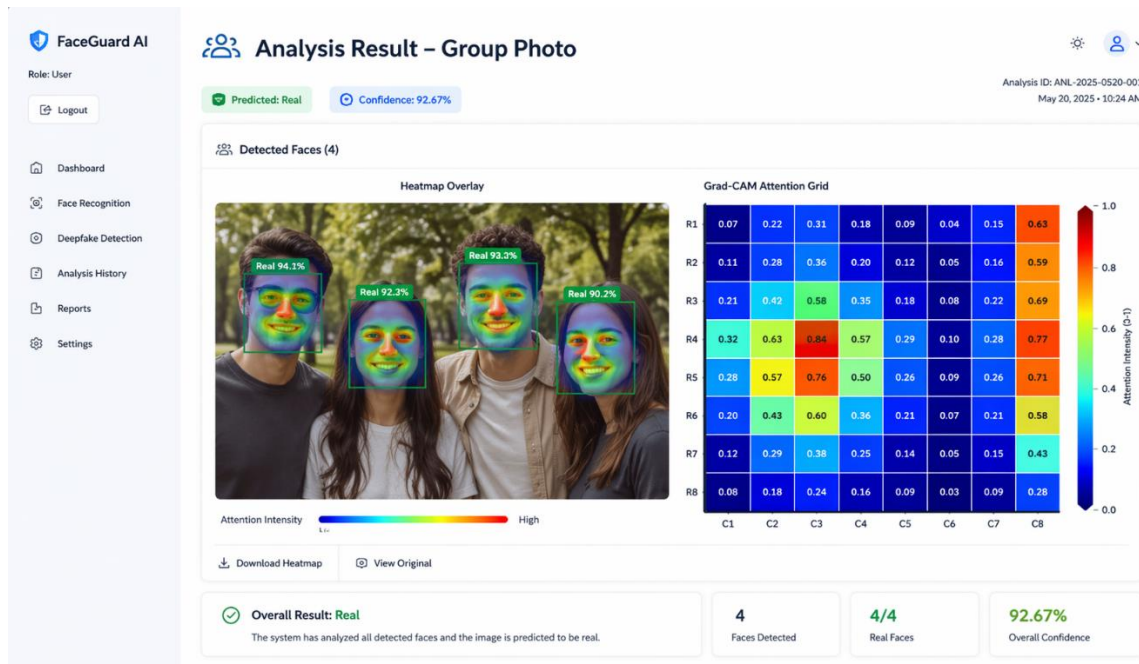


Fig-7.11 real image Output no.2(Heatmap)

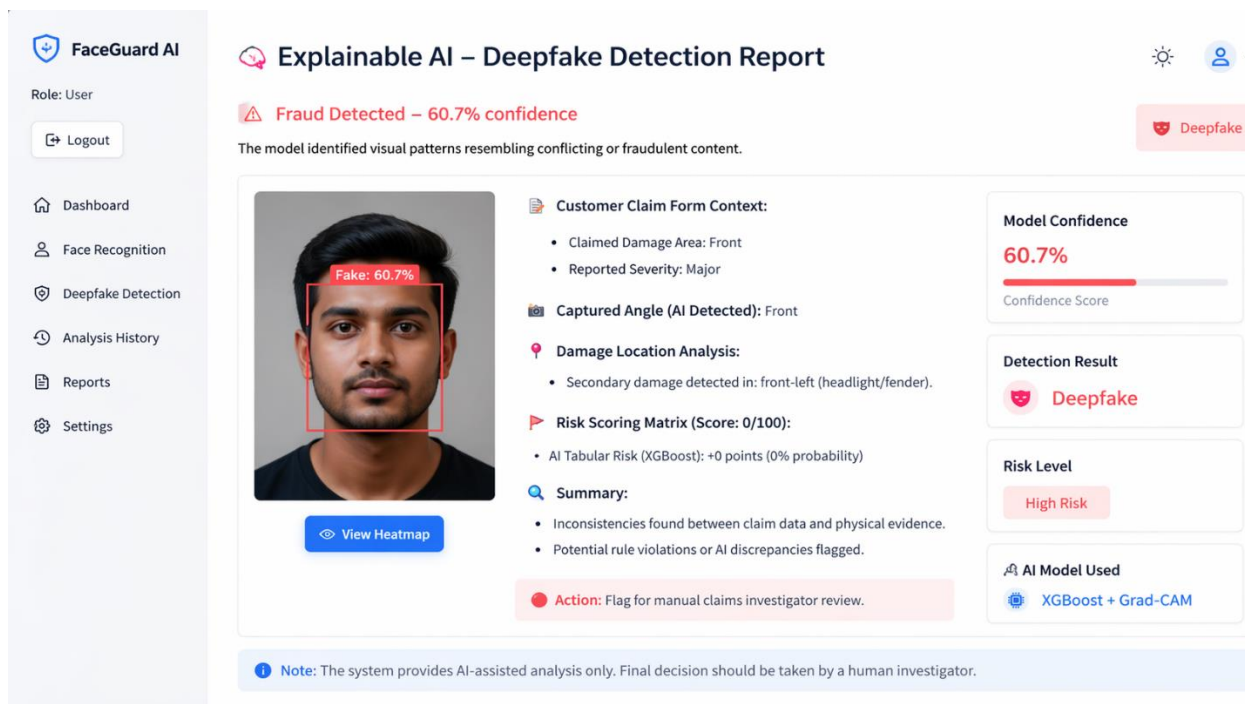


Fig-7.12 Fake image Output no.3(Explainable AI)

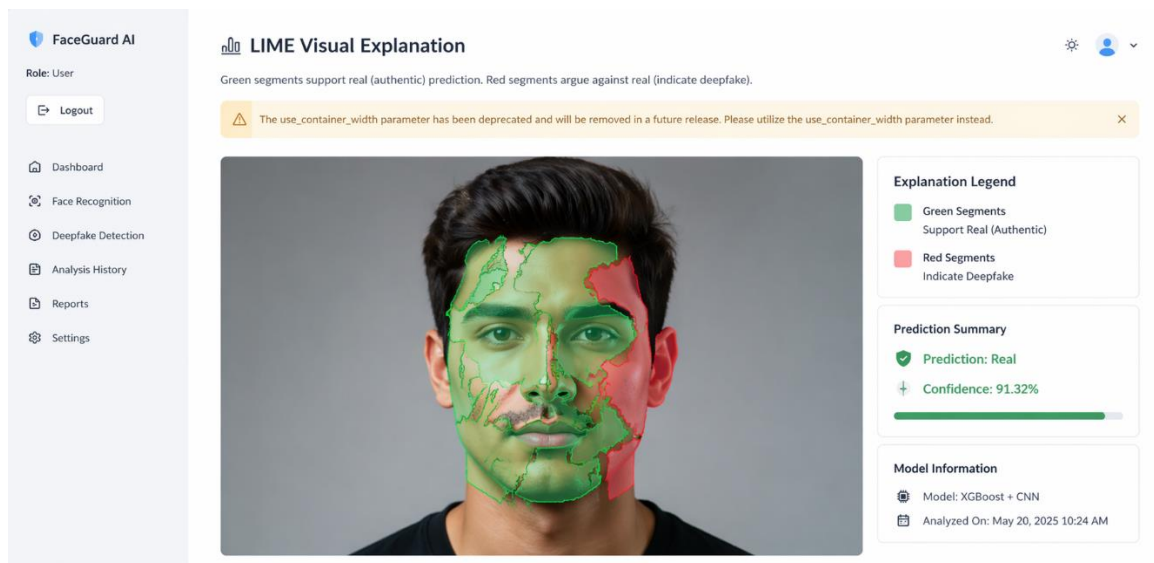


Fig-7.13 Fake image Output no.4(LIME)

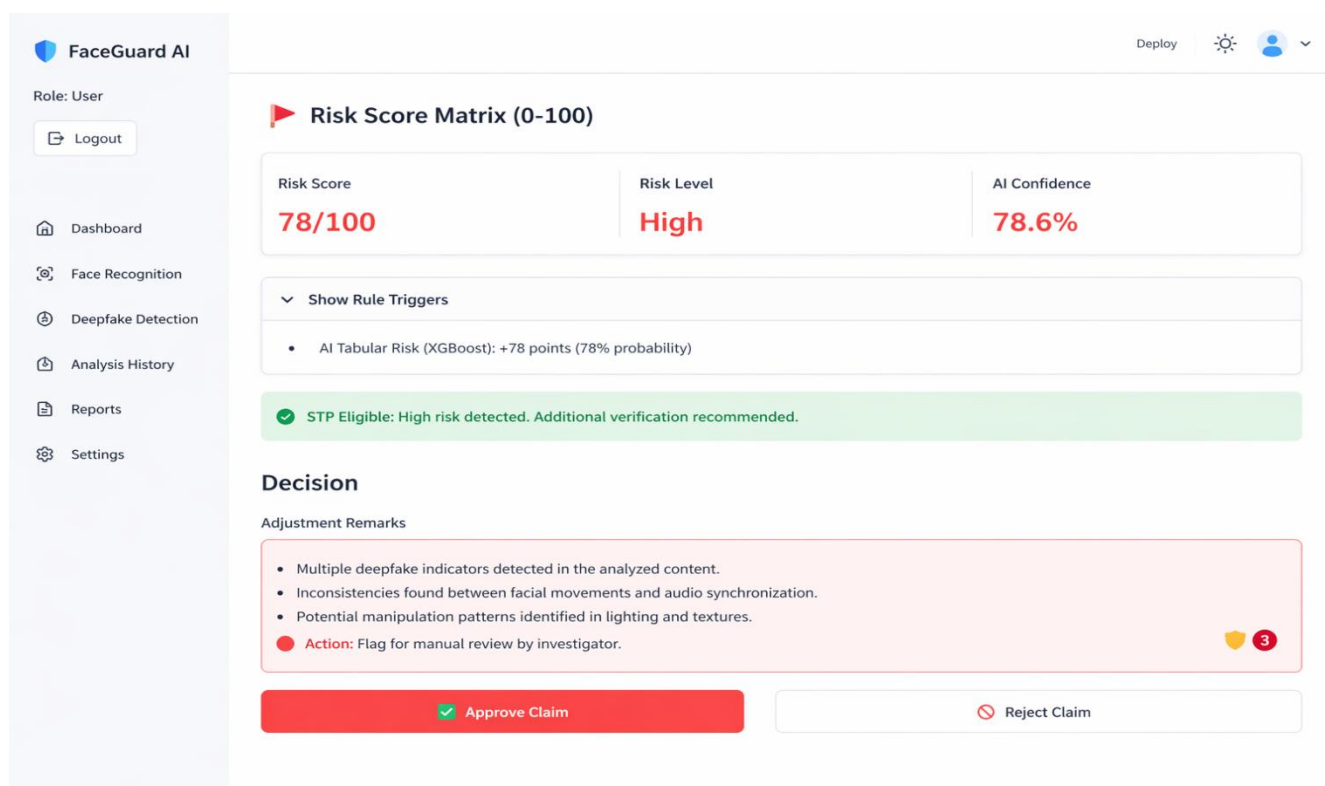


Fig-7.1 Fake image Output no.5(Risk Score Matrix & Decision)

The report demonstrates that the model maintains a good trade-off between detecting fraud and minimizing incorrect classifications

7.4 Confusion Matrix Discussion and Risk Scoring Interpretation

The confusion matrix plays a central role in evaluating the performance of the proposed system, as it provides a clear and detailed view of how accurately the model classifies images as real or deepfake. Instead of relying only on overall accuracy, the confusion matrix helps in understanding the distribution of correct and incorrect predictions across different classes.

It highlights four important outcomes: correctly identified real images, correctly detected deepfake images, and the cases where the system makes errors by either misclassifying real images as fake or failing to detect deepfake content. In the context of this project, missing a deepfake instance is more critical than incorrectly flagging a genuine image, since undetected manipulated content can lead to serious security and authenticity issues.

The evaluation process involves testing the trained model on unseen data and analyzing its predictions using performance metrics and the confusion matrix. This provides a deeper understanding of how well the system performs under real-world conditions and helps identify areas where improvements may be required.

In addition to classification performance, the system also incorporates a decision-support mechanism through a confidence-based interpretation layer. Instead of treating predictions as simple outputs, the system assigns a confidence score that reflects how strongly the model believes in its decision.

This confidence score is derived from the analysis of facial features and visual patterns detected during processing. Based on this value, the system categorizes the result into different levels such as low confidence, moderate confidence, or high confidence. This layered interpretation helps users better understand the reliability of the prediction.

By combining classification results with confidence-based interpretation, the system not only detects deepfake content but also provides meaningful insights that support informed decision-making. This approach enhances the overall effectiveness of the **system and makes it more suitable for practical applications.**

7.5 Visualization and Evaluation Insights

1. Training Curves

The training and validation curves provide a clear understanding of how the model learns over time. A steady decrease in loss indicates that the system is effectively improving during training. The curves remain stable without large fluctuations, which suggests that the model is not overfitting to the training data.

2. Confusion Matrix Heatmap

The confusion matrix heatmap offers a visual summary of the model's classification performance. It shows that most predictions match the actual labels, indicating strong model reliability.

3. Test Samples Visualization

The system was further evaluated using a set of unseen test images to analyze its real-world performance. The results demonstrate that the model can clearly distinguish between real and fake faces with meaningful confidence scores.

It performs well under different conditions such as changes in lighting, facial angles, and image quality. Even in cases where manipulations are subtle, the system is able to capture important inconsistencies and make accurate predictions.

7.6 Real-Time Deployment Results (Streamlit Application)

The deployed Streamlit application demonstrates the practical usability of the proposed system in real-time scenarios. When a user uploads an image, the system immediately processes it by detecting faces, extracting features, and analyzing whether the content is real or a deepfake.

allows the investigator to launch the AI analysis workflow. The system then performs image prediction, orientation detection, Grad-CAM visualization, LIME explanation, and hybrid risk scoring in a single operational flow. This section should emphasize practicality:

- The interface is responsive,
- The analysis is transparent,
- The interface is designed to be simple and user-friendly, allowing users to interact with the system without requiring technical expertise.

7.7 Discussion of Findings

The evaluation of the proposed Face Recognition and Deepfake Detection System highlights several important observations regarding its performance and practical usability.

- **Strong Detection Performance**
 - The system demonstrates a high ability to correctly recognize faces and identify deepfake content.
- **Handling of Data Variations**
 - The model performs well even when there are variations in lighting conditions, facial angles, and image quality
- **Explainability as a Key Advantage**
 - One of the major strengths of the system is its ability to provide understandable explanations for its predictions.
- **Reduction in Manual Effort**
 - The automated analysis significantly reduces the need for manual inspection.
- **Real-Time Decision Support**
 - The system delivers instant results through the deployed application.
- **Limitations**
 - The performance of the model depends heavily on the quality and diversity of the training data. Limited or biased datasets may affect accuracy
 - In some cases, the system may incorrectly classify real images as fake (false positives), which may require additional manual verification
 - The current dataset size may limit the system's ability to generalize across all possible deepfake techniques and variations

Chapter 8

8. Conclusions And Future Scope

8.1 Conclusions

The proposed Face Recognition and Deepfake Detection System represents a complete and practical implementation of an AI-based solution for identity verification and digital authenticity analysis. By combining computer vision techniques with machine learning approaches, the system effectively addresses the growing challenges of face recognition and detection of manipulated media.

The project successfully developed a **hybrid fraud detection framework** that combines:

- Face detection and recognition mechanisms for identifying individuals from images
- Deepfake detection techniques for distinguishing between real and manipulated facial content
- Feature extraction methods such as texture, shape, and color analysis to improve classification accuracy

This combined approach ensures that the system is capable of performing both identity recognition and authenticity verification within a single pipeline, making it more efficient and reliable.

One of the key strengths of the system is its ability to provide **interpretable results**.

Through visual outputs such as highlighted facial regions and attention-based analysis, users can clearly understand how the system arrives at its decisions. This transparency increases user confidence and makes the system more suitable for real-world applications.

The implementation of the system using a **Streamlit-based web application** further enhances its usability. The interface is designed to be simple and interactive, allowing users to upload images and receive results instantly. This enables::

- Easy interaction with the system
- Real-time face recognition and deepfake detection
- Clear visualization of results and confidence levels

From a performance standpoint, the system demonstrates:

- High accuracy in recognizing faces and detecting deepfake content
- Stable performance across different image conditions such as lighting and angles
- Consistent generalization when tested on unseen data

Overall, the project successfully delivers a scalable and efficient solution that combines accuracy, usability, and interpretability. It provides a strong foundation for applications in areas such as security systems, identity verification, and digital media authentication.

8.2 Future Scope

While the proposed system performs effectively within its current scope, there are several opportunities to further enhance its capabilities and extend its applicability in real-world scenarios. Future improvements can focus on increasing accuracy, scalability, and overall system intelligence..

1. Expansion of Dataset

The performance of the system largely depends on the quality and diversity of the dataset used during development. At present, the system is trained and tested on a limited set of facial images, including both real and deepfake samples.

In the future, the dataset can be expanded to include:

- A wider variety of faces from different age groups, ethnicities, and expressions
- Images captured under different lighting conditions, angles, and backgrounds
- More advanced and diverse deepfake examples

2. Enhanced Automation

Currently, the system provides predictions and confidence scores to assist users in decision-making. This can be further improved by introducing a higher level of automation.

For example:

- Images with high confidence of being real can be automatically accepted
- Images with strong indications of manipulation can be flagged immediately
- Borderline cases can be marked for manual review

3. Cloud-Based Deployment

The system is currently designed to run locally using a Streamlit application. To support larger-scale usage, it can be deployed on cloud platforms such as AWS, Google Cloud, or Azure.

Cloud deployment would provide several advantages:

- Ability to handle multiple users simultaneously
- Improved scalability and performance
- Secure storage and management of large datasets
- Integration with external systems and services

8.3 Summary

This project presents a complete end-to-end solution for face recognition and deepfake detection by effectively combining computer vision techniques with intelligent data processing methods. The system integrates multiple components, including face detection, feature extraction, identity recognition, and deepfake analysis, into a unified framework that works efficiently in real time.

The approach demonstrates how different techniques can be brought together to solve real-world problems related to digital identity verification and media authenticity. By analyzing facial features and identifying visual inconsistencies, the system is able to accurately recognize individuals and detect manipulated content.

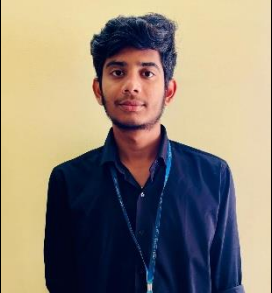
An important aspect of the system is its ability to provide clear and understandable outputs. The inclusion of confidence scores and visual indicators helps users interpret the results easily, improving transparency and trust in the system's decisions.

REFERENCES

- [1] I. Goodfellow et al., “Generative Adversarial Networks,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.
- [2] A. Rossler et al., “FaceForensics++: Learning to Detect Manipulated Facial Images,” *IEEE International Conference on Computer Vision (ICCV)*, 2019. IEEE
- [3] H. Nguyen et al., “Deep Learning for Deepfake Detection: A Survey,” *IEEE Access*, vol. 10, 2022.
- [4] F. Chollet, “Deep Learning with Python,” *Manning Publications*, 2018.
- [5] c M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems,” *Google Research*, 2016.
- [6] D. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” *International Conference on Learning Representations (ICLR)*, 2015.
- [7] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” *ICLR*, 2015.
- [8] M. Tan and Q. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *ICML*, 2019.
- [9] M. Tan and Q. Le, “EfficientNetV2: Smaller Models and Faster Training,” *Proceedings of Machine Learning Research (PMLR)*, 2021.
- [10] . Redmon et al., “You Only Look Once: Unified, Real-Time Object Detection,” *CVPR*, 2016.
- [11] K. Zhang et al., “Joint Face Detection and Alignment using Multi-task Cascaded Convolutional Networks,” *IEEE Signal Processing Letters*, 2016.
- [12] N. Dalal and B. Triggs, “Histograms of Oriented Gradients for Human Detection,” *CVPR*, 2005.
- [13] T. Ojala et al., “Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2002.
- [14] M. Ribeiro et al., “Why Should I Trust You? Explaining the Predictions of Any Classifier (LIME),” *KDD*, 2016.
- [15] R. Selvaraju et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” *ICCV*, 2017.

- [16] S. Afchar et al., “MesoNet: A Compact Facial Video Forgery Detection Network,” *IEEE International Workshop on Information Forensics and Security*, 2018.
- [17] Y. Li and S. Lyu, “Exposing DeepFake Videos by Detecting Face Warping Artifacts,” *CVPR Workshops*, 2019.
- [18] A. Agarwal et al., “Detecting Deepfake Videos with Temporal Inconsistencies,” *arXiv preprint*, 2020.
- [19] F. Schroff et al., “FaceNet: A Unified Embedding for Face Recognition and Clustering,” *CVPR*, 2015.
- [20] OpenCV Library, “Open Source Computer Vision Library,” <https://opencv.org>, accessed 2025.
- [21] Scikit-learn Developers, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, 2011.
- [22] Streamlit Inc., “Streamlit: The Fastest Way to Build Data Apps,” <https://streamlit.io>, accessed 2025.
- [23] NumPy Developers, “NumPy: Fundamental Package for Scientific Computing in Python,” <https://numpy.org>, accessed 2025.
- [24] Pandas Developers, “Pandas: Data Analysis and Manipulation Tool,” <https://pandas.pydata.org>, accessed 2025.
- [25] Matplotlib Developers, “Matplotlib: Visualization with Python,” <https://matplotlib.org>, accessed 2025.
- [26] Pillow Library, “Python Imaging Library (PIL Fork),” <https://python-pillow.org>, accessed 2025.

Information Regarding Student(s)

Student name	Email id	Permanent Address	Phone Number	Photograph
M.SAI SHREYAS REDDY	22btrcl094@jainuniversity.ac.in	Hindupur, Ananthapur, Andhra Pradesh	9390941673	
C.V.BHARG AV GOWD	22btrcl043@jainuniversity.ac.in	Hindupur, Ananthapur, Andhra Pradesh	9676926686	
K.HARSHA VARDHAN	22btrcl075@jainuniversity.ac.in	Hindupur, Ananthapur, Andhra Pradesh	8341902735	
C.MADHU KEERTHI	22btrcl038@jainuniversity.ac.in	Proddatur, Kadapa, Andhra Pradesh	9392725959	